Mohsin Iban Hossain

AIUB, Software Engineering Notes

SOFTWARE ENGINEERING

Table of Contents

SOFTWARE DESIGN

Ch: 9

Design

⇨ **Mitch Kapor's Software Design Principles (from his manifesto):**

1. **Firmness** – The software should be free from bugs that Interrupt its functionality.
2. **Commodity** – The software should serve its intended purpose effectively.
3. **Delight** – The software should provide a pleasurable user experience.

Or

⇨ **Mitch Kapor's Main Ideas on Good Software Design:**

1. **Firmness** – The program should not have bugs.
2. **Commodity** – The program should do what it is made for.
3. **Delight** – The program should be enjoyable to use.

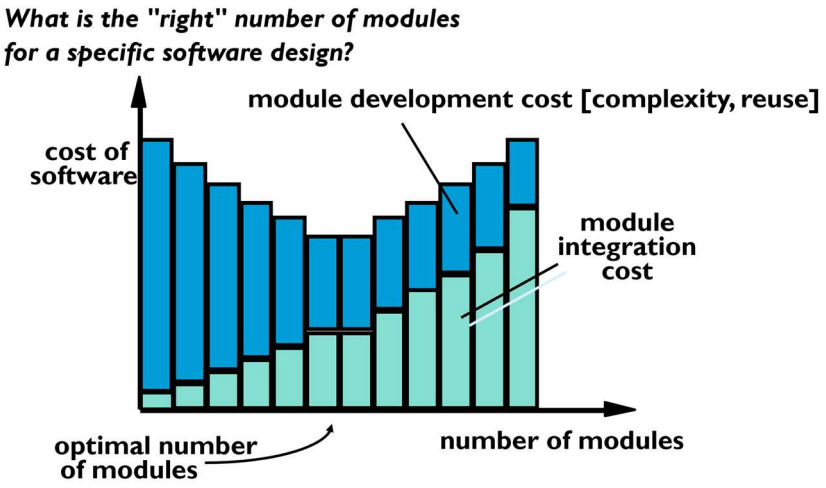
Modularity

1. **Modularity** means breaking a program into distinct, separate parts (modules).
2. It makes the program easier to understand and manage.
3. **Monolithic software** (one big block) is hard to understand and work with.
4. Breaking the program into modules reduces complexity and cost.

Functional Independence

1. **Cohesion** – Measures how well a module performs a single, focused task with minimal interaction with other modules.
2. **Coupling** – Measures the degree of interdependence between modules; lower coupling is better.
3. **Aspect** – Represents a **cross-cutting concern** that affects multiple parts of the program (e.g., logging or security).
4. **Refactoring** – Improves internal code structure **without changing its external behavior**.

Modularity: Trade-Offs



- The graph shows the **relationship between number of modules and software cost**.
- **Two types of cost**:
  - **Development cost** (**blue**) – decreases then increases with more modules.
  - **Integration cost** (**green**) – increases with more modules.
- **Total cost** is minimized at an **optimal number of modules**.
- Too few modules → high complexity.

  Too many modules → high integration effort.
- Goal: **Balance** both costs for efficient software design.

User Interface Design

User Interface Design – Key Goals:

1. **Easy to Learn**
2. **Easy to Use**
3. **Easy to Understand**

Typical Design Errors:

- **Lack of consistency**
- **Too much memorization**
- **No guidance / help**
- **No context sensitivity**
- **Obscure/ unfriendly**

Golden Rule - Place the user in control

1. **Flexible Interaction Modes** – Users should easily enter/exit modes without being forced into unwanted actions.
2. **Customization** – Allow flexibility in input (keyboard, mouse, pen, voice) and personalization (color, font, language).
3. **Interpretability & Undo** – Actions should be interruptible and reversible without data loss.
4. **Streamlined for All Skill Levels** – Support advanced users (e.g., macros) while remaining simple for beginners.
5. **Abstract Technical Details** – Hide system-level complexities; no need for O/S commands within the application.
6. **Direct Manipulation** – Enable interaction with on-screen objects as if they were real (e.g., drag-and-drop, progress bars).

Golden Rule – Reduce user's memory load

1. **Minimize Short-Term Memory Load** – Use visual cues to help users recognize, not recall, past actions.
2. **Meaningful Defaults** – Provide smart defaults with the option to customize and reset.
3. **Intuitive Shortcuts** – Use familiar, easy-to-remember shortcuts (e.g., Alt-P to print).
4. **Real-World Metaphors** – Use visual layout on real-world symbols (e.g., print icon = printer).
5. **Progressive Disclosure** – Present information in layers, starting from a high-level overview.

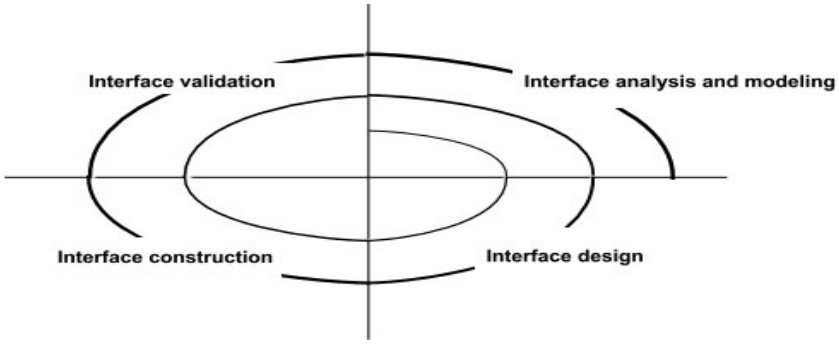
Golden Rule – Make the Interface Consistent

1. **Task Context Awareness** – Help users understand their current position and available next steps.
2. **Consistency Across Applications** – Maintain uniform behavior/design across related software (e.g., MS Office).
3. **Preserve User Expectations** – Avoid changing standard interaction patterns unless absolutely necessary.

Interface Analysis

⇨ **Interface Analysis Involves Understanding:**

1. **Users** – Who will use the system and their characteristics (skills, experience, needs).
2. **Tasks** – What actions users must perform to achieve their goals.
3. **Content** – What information is shown or required during interaction.
4. **Environment** – The physical or digital setting where the interface will be used (e.g., desktop, mobile, embedded systems).



User Analysis

➪ These questions are part of a **user analysis** in interface design. Here's a **condensed list of the key user-related factors** to consider:

1. **User Roles** – Are they professionals, technicians, officials, or factory workers?
2. **Education Level** – What is the typical educational background of users?
3. **Learning Preferences** – Can they learn from manuals, or do they prefer classroom training?
4. **Typing Skill** – Are they proficient typists or uncomfortable with keyboards?
5. **Demographics** – What are their age range and gender distribution?
6. **Work Environment** – Compensation model, work hours, and job demands.
7. **Usage Frequency** – Will the software be used daily or occasionally?
8. **Language** – What is the primary language spoken?
9. **Error Impact** – What are the risks or consequences of user mistakes?
10. **Domain Expertise** – Are they knowledgeable about the system's subject matter?
11. **Tech Curiosity** – Do users want to know about the technology the sits behind the interface?

Task Analysis & Modelling

➪ Answers the following questions …

1. What work will the user perform in specific circumstances?
2. What tasks and subtasks will be performed as the user does the work?
3. What specific problem domain objects will the user manipulate as work is performed?
4. What is the sequence of work tasks (hierarchy)—the workflow?

- **Use-Cases** – Define what users do in specific situations.
- **Task Elaboration** – Break down tasks into detailed steps and subtasks.
- **Object Elaboration** – Identify interface objects the user interacts with.
- **Workflow Analysis** – Show task sequences and roles (use swimlane diagrams).

Analysis of Display Content

1. **Consistent Data Placement** – Keep similar data (e.g., photos) in the same screen location.
2. **Report Partitioning** – Break large reports into sections for easier reading.
3. **Quick Access to Summary** – Provide shortcuts to summary info in large data sets.
4. **Responsive Graphics** – Scale visuals to fit the display (e.g., smartphones).
5. **Use of Color** – Apply color meaningfully (e.g., red for errors).
6. **Error Presentation** – Show errors/warnings via dialog boxes or messages.

INTERFACE DESIGN PRINCIPLES-I

1. **Anticipation** – Predict and prepare for the user's next action.
2. **Communication** – Show status feedback (e.g., progress bars).
3. **Consistency** – Use uniform navigation, icons, colors, and layout.
4. **Controlled Autonomy** – Guide users while enforcing navigation rules.
5. **Efficiency** – Optimize for user productivity, not just technical performance.

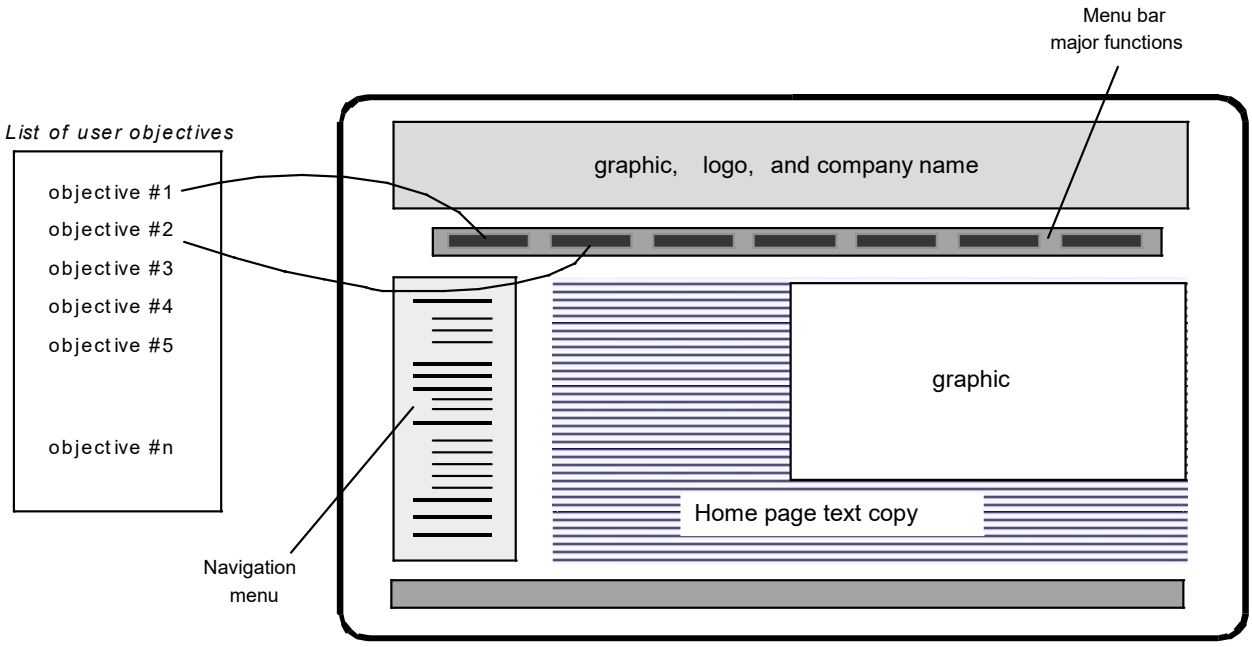
INTERFACE DESIGN PRINCIPLES-II

1. **Focus** – Keep the interface and content centered on the user's current tasks.
2. **Fitt's Law** – Target selection time depends on target size and distance.
3. **Human Interface Objects** – Use standard reusable UI components (e.g., Bootstrap).
4. **Latency Reduction** – Allow users to continue working during background tasks.
5. **Learnability** – Design for quick learning and easy relearn
6.

INTERFACE DESIGN PRINCIPLES-III

1. **Work Product Integrity** – Auto-save user input to prevent data loss.
2. **Readability** – Ensure all content is clear and easy to read.
3. **Track State** – Save user progress for seamless continuation later.
4. **Visible Navigation** – Make navigation intuitive without excessive scrolling.
5. **Use White Space** – Don't be afraid of white space
6. **Prioritize Content** – Focus on meaningful content over flashy design.
7. **Logical Layout** – Arrange elements top-left to bottom-right for natural flow.

Mapping user objectives (wareframming)



MCQ Practice

1. **Modularity in software design means**
   A) Testing software for bugs
   B) Dividing software into distinct logical parts (modules) that can be managed and recombined
   C) Writing all code in a single large module
   D) Designing the visual layout of the user interface
2. **Cohesion in a software module indicate?**
   A) Visual consistency of UI elements
   B) Refactoring code to improve performance
   C) How many modules are interconnected
   D) The degree to which a module performs a single, well-defined task with minimal interaction
3. **Why is refactoring important?**
   A) Designing user interface
   B) Testing software for bugs
   C) Adding new features
   D) Improving internal code structure without changing external behavior
4. **What is a key trade-off when deciding the number of modules?**
   A) Maximize modules for reuse regardless of integration cost
   B) Ignore module costs and focus on UI design
   C) Balance module integration cost and development cost
   D) Minimize modules to reduce development time
5. **Which is NOT a typical UI design error?**
   A) Providing too much guidance
   B) Lack of consistency
   C) No context sensitivity
   D) Excessive memorization required
6. **According to 'Place the user in control', what should be done?**
   A) Force users into specific modes
   B) Hide interaction options
   C) Prevent undo actions
   D) Allow easy mode switching and flexible interaction
7. **'Reduce user's memory load' mean?**
   A) Use unfamiliar metaphors
   B) Provide visual cues and meaningful defaults
   C) Eliminate shortcuts
   D) Require memorization of commands
8. **Why is consistency important in UI design?**
   A) Users memorize all elements
   B) Helps users understand context and leverages familiar models
   C) Use random colors and layouts
   D) Force users to learn new models every time

9. **Interface analysis involve?**
   A) Testing software bugs
   B) Writing backend code
   C) Understanding users, tasks, content, and environment
   D) Designing graphical elements
10. **Which question is part of user analysis?**
   A) Number of modules
   B) Color scheme
   C) Programming language
   D) Are users expert typists?
11. **The purpose of use-cases in task analysis?**
   A) Design visual layout
   B) Test performance
   C) Write source code
   D) Define basic user-system interactions
12. **How should display content be analyzed?**
   A) Avoid color
   B) Present all info at once
   C) Random placement
   D) Assign data to consistent locations and use color for clarity
13. **Why is font choice significant?**
   A) Fonts carry a message and affect user perception
   B) Always use decorative fonts
   C) Use smallest fonts possible
   D) Fonts are irrelevant
14. **Anticipation in interface design mean?**
   A) Predict and prepare for user's next move
   B) Use consistent colors
   C) Minimize delays
   D) Focus on current task
15. **How does Fitt's Law influence design?**
   A) Color choices don't matter
   B) Time to acquire target depends on distance and size
   C) Use only keyboard shortcuts
   D) Users should memorize commands
16. **Why maintain work product integrity?**
   A) Hide errors
   B) Auto-save user work to prevent loss
   C) Allow code edits by users
   D) Force task completion without saving

17. **Why is visible navigation important?**
    A) Hide controls
    B) Force scrolling
    C) Provide sense of place and easy movement
    D) Require memorization
18. **Wireframing's role in user objectives?**
    A) Conduct training
    B) Organize user objectives and interface elements visually before development
    C) Test performance
    D) Write backend code
19. **How does coupling differ from cohesion?**
    A) Coupling = module focus, cohesion = interface complexity
    B) Coupling = interdependence between modules, cohesion = module's single task focus
    C) Both refer to module count
    D) Coupling = UI design, cohesion = code testing
20. **Which of these is a key quality of good software design according to Mitch Kapor?**
    A) Complexity
    B) Firmness (no bugs)
    C) High coupling
    D) Monolithic structure

21. What is the primary benefit of designing a software module with high cohesion?

A) High cohesion refers to the physical proximity of code files on the disk.

B) A highly cohesive module interacts extensively with other modules to share responsibilities.

C) A highly cohesive module performs a single task, requiring little interaction with other modules, making it easier to maintain and understand.

D) High cohesion means a module contains many unrelated functions to improve flexibility.

22. What Why is coupling an important consideration in software design?

A. Coupling measures the speed of data transfer within a module.

B. Coupling indicates the degree of interdependence between modules, affecting maintainability and complexity.

C. Coupling is the process of integrating user interfaces with databases.

D. Coupling refers to the number of lines of code in a module.

SOFTWARE TESTING

Ch: 10

Software Testing

1. **Purpose** – Testing is done to find errors before delivering to users.
2. **Testability** – Refers to how easily a program can be tested.
3. **Testing Reveals:**
    - Errors
    - Requirement's conformance
    - Performance issues
    - Overall quality indicator

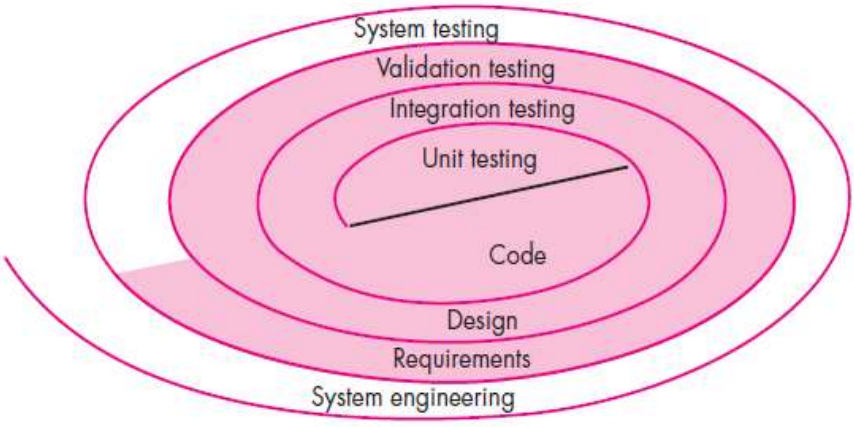
Who Tests the Software?

- **Developer:**
    - Knows the system
    - Tests gently
    - Focused on delivery
    - Operates in a familiar environment
- **Independent Tester:**
    - Learns the system
    - Tries to break it
    - Focused on quality
    - Tests from a fresh, unbiased perspective

V & V

- **Validation** – Ensures the software meets **customer requirements** ("Are we building the right product?").
- **Verification** – Ensures the software is **correctly implemented** ("Are we building the product right?").

Testing Strategy



- Start with **testing-in-the-small**, then move to **testing-in-the-large**.
- **Conventional software:**
  - Start with modules → then integrate them.
- **Object-Oriented (OO) software:**
  - Start with **classes** (attributes + operations + interactions).

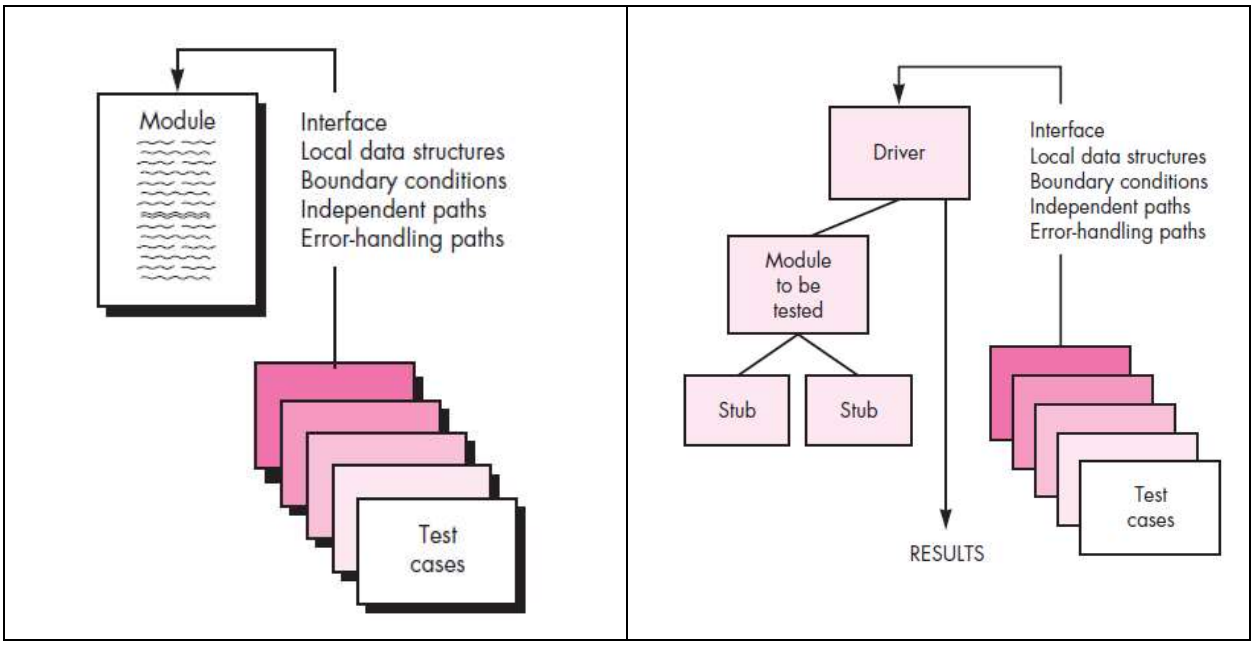
Testing Strategic Issues

1. **Define Requirements Clearly** – Use measurable terms early.
2. **Set Clear Testing Objectives** – Know what you aim to test.
3. **Understand Users** – Create user profiles.
4. **Use Rapid Cycle Testing** – Test frequently in short iterations.
5. **Build Self-Testing Software** – Design for internal checks.
6. **Do Technical Reviews Early** – Catch errors before testing.
7. **Review Test Strategy** – Validate test plans and cases.
8. **Focus on Continuous Improvement** – Evolve the testing process.

Unit Testing

➔ **Unit Testing** is the process of testing individual components or functions of a software application in isolation to ensure they work correctly. It is the first level of testing done during development.

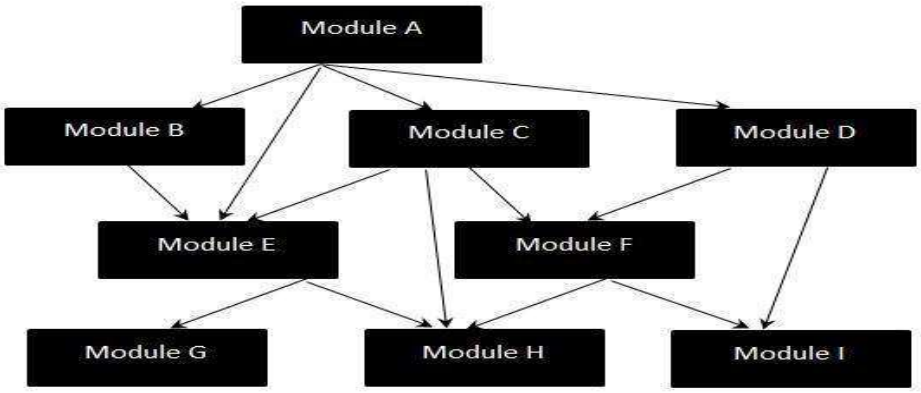
- **Unit Testing** – Tests individual software components (units).
- **Performed By** – Typically done by the programmer.
- **Before** – Conducted prior to integration testing.
- **Stub** – A placeholder simulating lower-level modules not yet developed or integrated.



Integration Testing Strategies

➔ **Integration Testing** is the process of testing how different software modules work together. It checks the interaction between units to ensure they function as a complete system.

1. **System Integration Testing (SIT):**
   - Combines modules systematically.
   - Focuses on finding interface-related errors.
2. **Big Bang Approach:**
   - All modules integrated at once.
   - Entire system is tested together.
   - Hard to isolate issues; used less often.



Top-down Integration

➔ **Top-Down Integration Testing** is a strategy where testing starts from the top-level modules and moves down to the lower-level modules step by step.

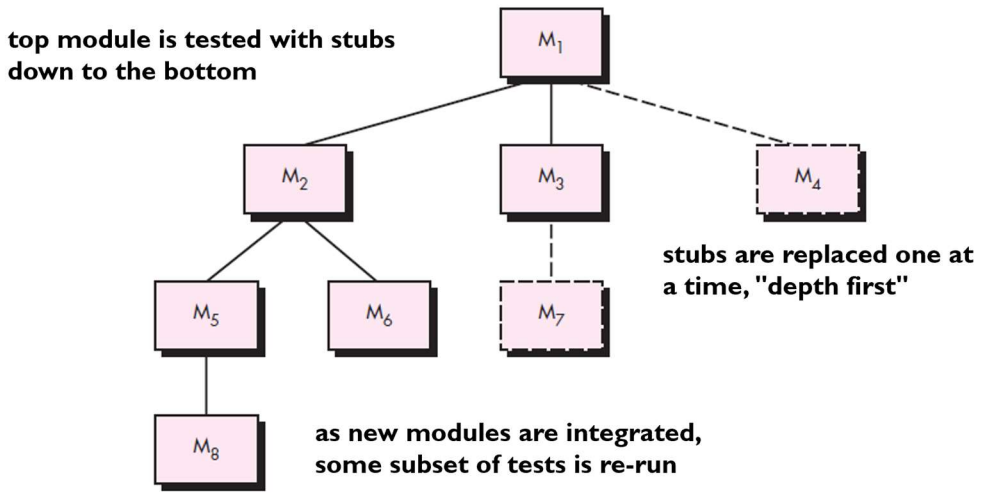
- High-level modules are tested first.
- Lower-level modules are integrated and tested one at a time.
- **Stubs** are used to simulate lower-level modules not yet developed.

Advantages:

- Early testing of high-level logic.
- Major design flaws can be identified early.

Disadvantages:

- Lower-level modules are tested late.
- Requires many stubs, which can be time-consuming to create.



Bottom-up Integration

➔ **Bottom-Up Integration Testing** is a strategy where testing begins with the lower-level (leaf) modules and progresses upward to higher-level modules.

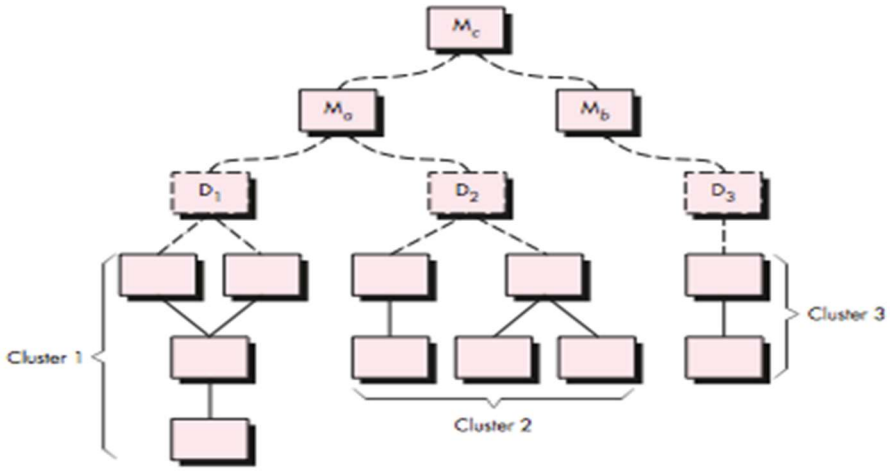
- Low-level modules are tested first.
- Higher-level modules are integrated one at a time.
- **Drivers** are used to simulate higher-level modules not yet developed.

Advantages:

- Low-level functionalities are tested early.
- Easier to identify issues in foundational code.

Disadvantages:

- High-level logic is tested late.
- Requires many drivers, which can be complex to build.



Regression Testing

➔ **Regression Testing** is the process of re-testing a software system after changes (like bug fixes or new features) to ensure that the existing functionality still works correctly.

- Verifies that updates haven't introduced new bugs.
- Often automated for efficiency.
- Performed after code changes, enhancements, or configuration updates.

Example:

If a login bug is fixed, regression testing checks that other features (like registration or profile update) still work properly.

Main Points on Regression Testing:

1. **Purpose:** Re-run previous tests to check for unintended side effects after changes.
2. **When:** After bug fixes, updates, or configuration changes.
3. **Benefit:** Ensures new changes don't break existing functionality.
4. **Execution:** Can be done manually or with automated tools.

Smoke Testing

➔ A quick, basic check of a new software build to ensure critical functions work and the build is stable enough for detailed testing.

- Also called **Build Verification Testing**
- Tests major features superficially, not in-depth
- Done before full testing to catch major issues early
- Saves time by rejecting broken builds upfront

Example:

Check if the app launches, login works, and main pages load after a new build.

Smoke Testing Steps:

1. **Daily Build Integration** – Code is compiled into a full build every day.
2. **Includes All Components** – Data files, libraries, modules, etc.
3. **Initial Tests Run** – Detect critical ("show stopper") errors early.
4. **Goal** – Ensure basic functionality works; avoid major failures.
5. **Integration** – Can be **top-down** or **bottom-up**.
6. **Frequency** – Smoke testing is performed **daily**.

Object-Oriented Testing

- **Class Testing = Unit Testing** for object-oriented programs.
- **Tests:**
  - Individual **operations** (methods)
  - **State behavior** of the class

Integration Strategies:

1. **Thread-Based Testing** – Test classes responding to a single input/event.
2. **Use-Based Testing** – Test classes involved in a specific use case.
3. **Cluster Testing** – Test groups of classes that collaborate together.

Higher Order Testing

1. **System Testing:**
   - Tests full system integration (hardware, OS, software compatibility).
2. **Alpha Testing:**
   - Done by users or testers at the developer's site.
   - Internal acceptance test before public release.
3. **Beta Testing:**
   - Done by real users in a real environment.
   - Helps gather feedback before final release.
4. **Recovery Testing:**
   - Forces failures to verify proper system recovery.
5. **Security Testing:**
   - Ensures system is protected from unauthorized access.
6. **Stress Testing:**
   - Tests system under extreme load or conditions.
7. **Performance Testing:**
   - Measures system performance (e.g., response time, throughput).

Debugging

➔ The process of identifying, analyzing, and **fixing bugs** or defects in software code.

- Done after finding errors during testing or use
- Involves locating the root cause of a problem
- Fixes the code to make the software work correctly
- Can use tools like debuggers, logs, and breakpoints

Example:
A developer finds why the login fails and corrects the faulty code.

Main Points on Debugging Process:

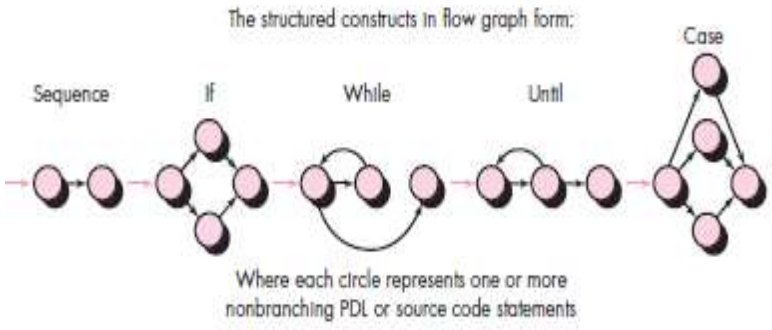
- Debugging matches **symptom** with **underlying cause**.
- A **symptom** may vanish when another issue is fixed.
- The **cause** might:
  - Be a combination of **non-errors**.
  - Stem from **system or compiler errors**.
  - Be due to **false assumptions** accepted by everyone.

Debugging Techniques

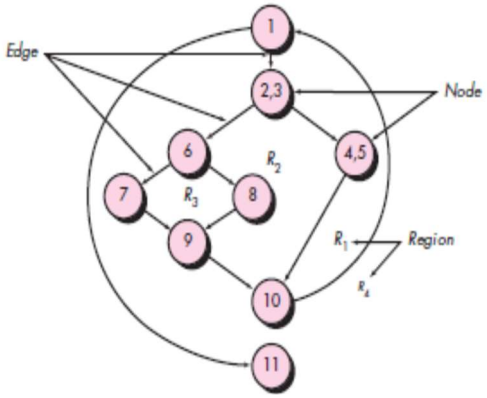
- **Brute Force Testing**:
  - Most common, least efficient
  - Uses memory dumps, runtime traces, output statements
- **Backtracking**:
  - Trace source code **backward** manually
  - Best for **small programs**
- **Cause Elimination**:
  - Form a **hypothesis** about the cause
  - Test and **refine data** to isolate the bug

Basis-path Testing

- **Basis Path Method**:
  - Helps measure **logical complexity** of code
  - Guides creation of a **basis set of test paths**
- **McCabe's Approach**:
  - Represents program as a **directed graph**
  - **Nodes** = program statements
  - **Edges** = control flow between statements



Independent Program Paths



Path 1: 1-11
Path 2: 1-2-3-4-5-10-1-11
Path 3: 1-2-3-6-8-9-10-1-11
Path 4: 1-2-3-6-7-9-10-1-11

Each **path** introduces **independent**

Paths like `1-2-3-4-5-10-1-2-3-6-8-9-10-1-11` are **not independent** because they **reuse existing edges**.

Cyclomatic Complexity (V(G))

It provides the **number of independent paths** through the program.

Formula:

Method-01:

$$V(G) = E - N + 2P$$

Where:
E = number of edges
N = number of nodes
P = number of connected components (usually 1 for a single program)

Method-02:

$$V(G) = \text{number of predicate nodes} + 1$$

This number tells you **how many paths** you need to design for complete path coverage.

Method-03:

$$V(G) = \text{Total number of closed regions in the control flow graph} + 1$$

Cyclomatic Complexity

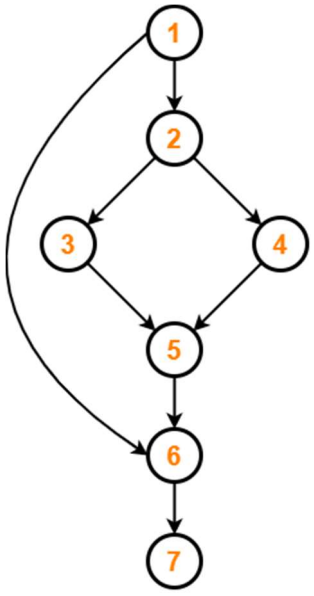
- Measures **logical complexity** of a program.
- Can be computed in three ways:

1. **Number of independent paths** in the program.
2. **Number of regions** in the flow graph (e.g., 4 regions).
3. Using formulas on the flow graph G:
    - $\mathbf{V(G) = E - N + 2}$
        - E = number of edges
        - N = number of nodes
        - Example: $\mathbf{11 - 9 + 2 = 4}$
    - **Or, V(G)=P+1**
        - P = number of predicate (decision) nodes
        - Example: $\mathbf{3 + 1 = 4}$ where predicate nodes are 1, 2, 3, 6

- Cyclomatic complexity value indicates **how many test paths** are needed.

Practice Problem of Cyclomatic Complexity

Problem1: Calculate cyclomatic complexity for the



Control Flow Graph

Method-03:

$$\mathbf{V(G) = Total\ number\ of\ closed\ regions\ in\ the\ control\ flow\ graph + 1}$$

$$\therefore V(G) = 2 + 1 = 3$$

Method-01:

$$\mathbf{V(G) = E - N + 2}$$

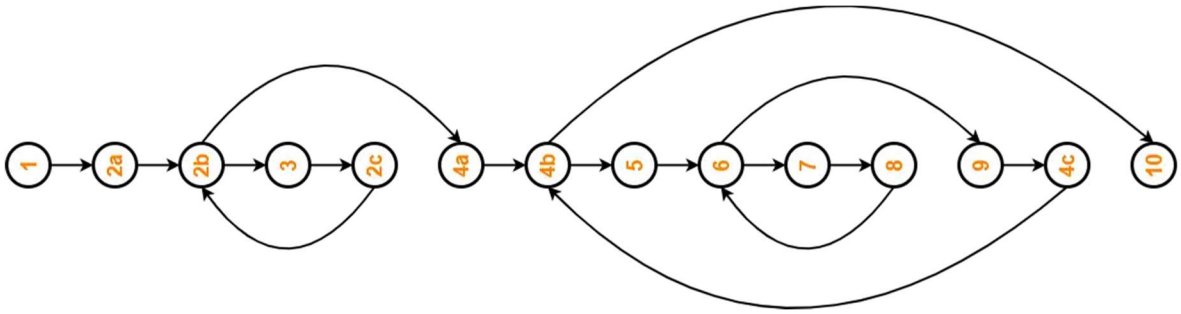
$$\therefore V(G) = 8 - 7 + 2 = 3$$

Method-02:

$$\mathbf{V(G) = P + 1}$$

$$\therefore V(G) = 2 + 1 = 3$$

Problem2: Calculate cyclomatic complexity for the



Method-03:

$$\mathbf{V(G) = Total\ number\ of\ closed\ regions\ in\ the\ control\ flow\ graph + 1}$$

$$\therefore V(G) = 3 + 1 = 4$$

Method-01:

$$\mathbf{V(G) = E - N + 2}$$

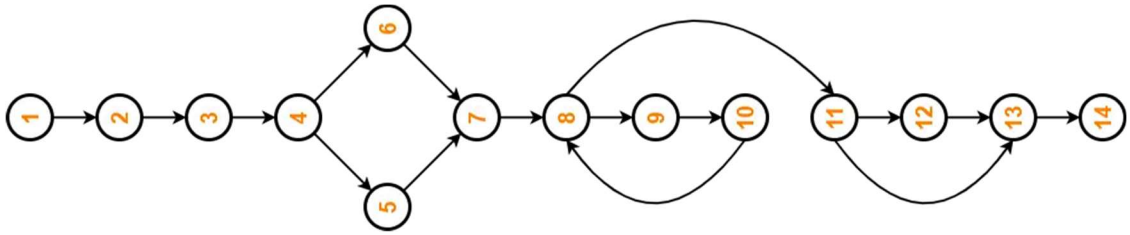
$$\therefore V(G) = 16 - 14 + 2 = 4$$

Method-02:

$$\mathbf{V(G) = P + 1}$$

$$\therefore V(G) = 3 + 1 = 4$$

Problem3: Calculate cyclomatic complexity for the



Method-03:

$$\mathbf{V(G)} = \textbf{\textit{Total number of closed regions in the control flow graph}} + \mathbf{1}$$

$$\therefore V(G) = 3 + 1 = 4$$

Method-01:

$$\mathbf{V(G)} = \mathbf{E - N + 2}$$

$$\therefore V(G) = 16 - 14 + 2 = 4$$

Method-02:

$$\mathbf{V(G)} = \mathbf{P + 1}$$

$$\therefore V(G) = 3 + 1 = 4$$

White-Box Testing

- Test all **independent paths** in a module at least once.
- Exercise all **logical decisions** with both true and false outcomes.
- Test all **loops** at boundaries and within their limits.
- Validate **internal data structures** to ensure correctness.

Black-Box Testing

- Based on **functional requirements** of the software.
- **Detects errors like:**
  - Incorrect or missing functions
  - Interface errors
  - External database access errors
  - Initialization and termination errors

MCQ Practice

1. What is the primary intent of software testing before delivery to the end user?
a) Verify performance benchmarks
b) Find errors prior to delivery
c) Ensure user-friendliness
d) Validate documentation

2. How does software testability affect the testing process?
a) Determines ease of testing
b) Defines number of test cases
c) Measures performance
d) Specifies security vulnerabilities

3. What is the key difference between a developer testing software and an independent tester?
a) Developer tests gently, driven by delivery; tester tries to break software, driven by quality
b) Developer tests after integration; tester before integration
c) Developer focuses on performance; tester on security
d) Developer tests UI only; tester tests backend

4. How do validation and verification differ?
a) Validation checks syntax; verification checks semantics
b) Validation ensures customer requirements; verification ensures correct implementation
c) Validation done by developers; verification by users
d) Validation tests performance; verification tests security

5. What is the main focus when "testing-in-the-small" for object-oriented software?
a) System integration
b) Individual functions only
c) Testing an OO class with attributes and operations
d) UI components only

6. Which is NOT a strategic issue in planning software testing?
a) Rapid cycle testing
b) Explicit objectives
c) Ignoring user profiles
d) Quantifiable requirements

7. What is the role of a stub in unit testing?
a) Testing report
b) Simulates lower-level modules
c) Integration technique
d) Automation tool

8. What characterizes the 'big bang' integration testing?
a) Integrate top module first
b) Integrate bottom modules first
c) Integrate subset of modules
d) Integrate all units at once

9. How are stubs used in top-down integration testing?
a) Test top module with stubs, replace stubs gradually
b) Stubs used after full integration
c) Stubs simulate user inputs
d) Stubs replace top module

10. What is the main purpose of regression testing?
a) Test units before integration
b) Validate UI requirements
c) Test under extreme load
d) Re-execute tests to check for unintended side effects

11. What is the primary goal of smoke testing?
a) Validate security
b) Test performance under load
c) Find critical errors blocking build
d) Exhaustive feature testing

12. In object-oriented testing, what does thread-based testing focus on?
a) Attributes only
b) Classes responding to one input/event
c) Independent class testing
d) Entire system testing

13. Which best describes alpha testing?
a) Testing by end users post-release
b) Internal testing by potential users before beta
c) Recovery testing
d) Automated module testing

14. What is the main challenge addressed by debugging?
a) Meeting customer requirements
b) Measuring performance
c) Automating test execution
d) Matching symptoms to causes

15. Which debugging technique traces source code backward manually?
a) Cause elimination
b) Automated testing
c) Backtracking
d) Brute force testing

16. What does basis-path testing help determine?
a) Security vulnerabilities
b) Performance bottlenecks
c) Number of UI elements
d) Logical complexity and test paths

17. How is cyclomatic complexity calculated?
a) Measure execution time
b) Count user inputs
c) V(G) = E - N + 2 (edges - nodes + 2)
d) Count lines of code

18. Which is NOT a goal of white-box testing?
a) Execute loops at boundaries
b) Exercise logical decisions
c) Cover all independent paths
d) Test UI compliance

19. What errors does black-box testing primarily find?

a) Internal data structure errors

b) Compiler errors

c) Missing functions, interface, database, behavior, initialization errors

d) Syntax errors

20. What is the difference between alpha and beta testing?

a) Alpha by end users; beta internal

b) Alpha internal before release; beta by users after alpha

c) Alpha for recovery; beta for performance

d) Alpha automated; beta manual

True/False Practice

1. Testing is performed to find errors before software delivery. True

2. Software testability refers to how difficult it is to test a program. False

3. Developers tend to test software aggressively to find faults. False

4. Validation ensures the software meets customer requirements. True

5. Integration testing always starts from the bottom modules. False

6. Regression testing ensures changes do not introduce new errors. True

7. Smoke testing is a detailed and exhaustive test of all software features. False

8. Stubs simulate higher-level modules during integration testing. False

9. Cyclomatic complexity measures the number of independent paths in a program. True

10. Black-box testing requires knowledge of internal code structure. False

SOFTWARE QUALITY ATTRIBUTES

Ch: 11

Software Quality Attributes

Overview of Software Quality Requirements:

- Software success depends on more than just functionality.
- Quality attributes (nonfunctional requirements) define overall system performance and user experience.
- Key quality factors include:
  - **Usability** – Ease of use.
  - **Performance** – Speed and responsiveness.
  - **Reliability** – Frequency of failures.
  - **Robustness** – Handling of unexpected conditions.
- Customers rarely state quality expectations clearly.
- Terms like **user-friendly**, **fast**, **reliable**, and **robust** need detailed interpretation.
- Quality attributes heavily influence **architecture and design**.
- It's easier and cheaper to **design for quality early** than to fix it later.

Availability

- **Availability** =

$$\frac{\text{Mean Time to Failure (MTTF)}}{\text{MTTF} + \text{Mean Time to Repair(MTTR)}}$$

- It measures how often the system is operational and ready for use.
- **Critical for web and global applications** due to continuous user access.
- **Example Requirement:**
  - 99.5% availability on weekdays (6:00 a.m. – midnight)
  - 99.95% availability during peak hours (4:00 p.m. – 6:00 p.m.)

Performance

Key Points on Performance Requirements:

- Define **how well and how fast** the system performs tasks.
- Key aspects:
  - **Speed** (e.g., database response time)
  - **Throughput** (e.g., transactions per second)
  - **Capacity** (e.g., number of concurrent users)
  - **Timing** (e.g., real-time constraints)
- Must address **performance under overload** conditions.
- **Examples:**
  - Web pages load ≤ 15 seconds over 50 KBps.
  - ATM withdrawal authorization ≤ 10 seconds.

Efficiency

Key Points on Efficiency Requirements:

- **Efficiency** measures how well the system uses resources like **CPU, memory, disk, and bandwidth**.
- Closely related to **performance** (e.g., response time).
- **Inefficient systems** lead to poor performance and may pose **safety risks** in real-time systems.
- Define goals considering **minimum hardware configurations**.
- **Example:**
  - 25% of CPU and RAM must remain unused at peak load.
- Users rarely express these needs directly—**analysts must extract them** through targeted questions.

Flexibility

Key Points on Flexibility Requirements:

- **Flexibility** = ease of adding new features or enhancements.
- Also called **extensibility**, **expandability**, **augmentability**.
- Important for **iterative development** or **evolutionary prototyping**.
- Encourages design choices that support **future changes**.
- **Example:**
    - A programmer with 6+ months' experience can add new output in **≤ 1 hour** (including coding and testing).

Integrity

Key Points on Integrity Requirements:

- **Integrity** (includes **security**) ensures:
    - No **unauthorized access**
    - **Data protection** from loss, viruses, or leaks
    - **Privacy and safety** of user data
- Critical for **Internet and e-commerce systems**
- Must be defined with **zero tolerance for error**
- Clearly specify:
    - **User authentication**
    - **Access control**
    - **Protected data**
- **Example:**
    - Only users with **Auditor** privileges can view **customer transaction histories**.

Interoperability

Key Points on Interoperability Requirements:

- **Interoperability** = ease of **data/service exchange** with other systems.
- Requires knowing:
  - **Related applications** users work with.
  - **Data types** to be exchanged.
- Common in systems like **SIM-NID integration** or **supermarket one-stop services**.
- **Example:**
  - Chemical Tracking System must **import chemical structures** from **ChemiDraw v2.3** and **Chem-Struct v5** or earlier.

Reliability

Key Points on Reliability Requirements:

- **Reliability** = probability of software running **without failure** over time.
- Measured by:
  - **Success rate** of operations
  - **Mean time between failures (MTBF)**
- Requirements depend on **failure impact** and **cost justification**.
- High-reliability systems need **high testability**.
- **Example:**
  - No more than **5 failures per 1000** experimental runs.

Robustness

Key Points on Robustness Requirements:

- **Robustness** = ability to keep functioning despite:
  - Invalid inputs
  - Software/hardware defects
  - Unexpected conditions
- Robust systems **recover gracefully** and handle user errors forgivingly.

- Also known as **fault tolerance**.
- Elicit requirements by asking users about possible **error scenarios** and desired system responses.
- **Example:**
    - Editor must **recover changes made up to 1 minute before a crash** on next start.

Usability

Key Points on Usability Requirements:

- **Usability** measures the effort needed to:
    - Prepare inputs
    - Operate the system
    - Understand outputs
- Also called **ease of use** or **human engineering**.
- Includes **ease of learning** for new/infrequent users.
- Usability goals can be **quantified and measured**.
- **Example:**
    - Trained user can submit a chemical request in **4 to 6 minutes** on average.

Maintainability

- **Maintainability** = ease of **fixing defects** and **modifying software**.
- Depends on how easily the software can be **understood, changed, and tested**.
- Closely linked to **flexibility** and **testability**.
- Critical for frequently updated or fast-developed products.
- Measured by:
    - Average time to fix issues
    - Percentage of successful fixes
- **Example:**
    - Maintenance programmer can update reports to new regulations in **20 hours or less**.

Reusability

Key Points on Reusability Requirements:

- **Reusability** measures the effort to adapt software components for use in other applications.
- Creating reusable components requires **more development effort**.
- Reusable software should be:
  - **Modular**
  - **Well documented**
  - **Independent** of specific apps and environments
  - **Generic** in functionality
- Reusability is **hard to quantify**.
- **Example:**
  - Chemical structure input functions must be reusable at the object code level in other apps using international chemical standards.

Testability

Key Points on Testability Requirements:

- **Testability** = ease of testing software components for defects.
- Also called **verifiability**.
- Crucial for products with **complex algorithms** or **ambiguous functionality**.
- Important for frequently modified products due to **regression testing** needs.
- **Example:**
  - Maximum **cyclomatic complexity** per module shall not exceed **20** (limits logic branches).

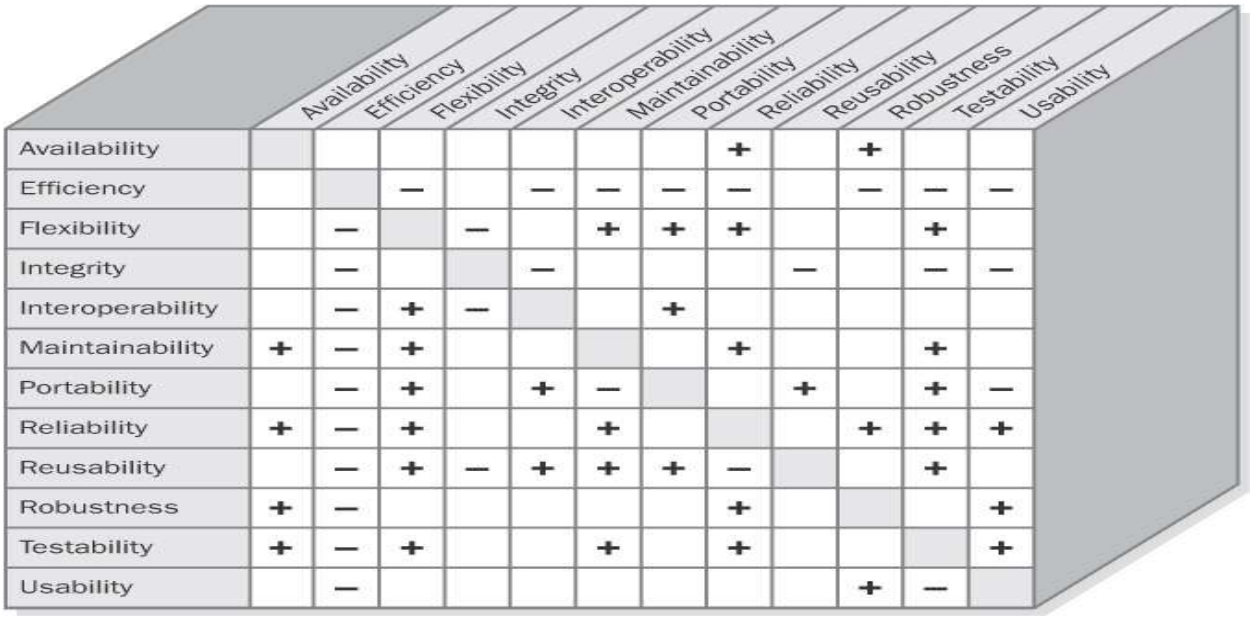
Quality Attributes

- Different product parts require different **quality attribute priorities**.
- Examples of critical attributes by system type:
  - **Embedded systems:** efficiency, reliability, safety, installability, serviceability
  - **Internet & mainframe apps:** availability, integrity, maintainability, scalability
  - **Desktop systems:** interoperability, usability

Quality Attributes to Who?

Important Primarily to Users	Important Primarily to Developers
Availability	Maintainability
Efficiency	Portability (runs on multiple platforms)
Flexibility	Reusability
Integrity	Testability
Interoperability	
Reliability	
Robustness	
Usability	

Attribute Trade-Offs



Key Points on Quality Attribute Trade-offs:

- **"+" (Plus sign):** Improving the row attribute **positively affects** the column attribute.
- **"–" (Minus sign):** Improving the row attribute **negatively affects** the column attribute.
- **Blank cell:** Little or no impact between attributes.
- Increasing **portability** often boosts:
  - Flexibility
  - Interoperability
  - Reusability
  - Testability
- Focusing on **usability**, **flexibility**, **reusability**, and **interoperability** can **reduce performance**.

Attribute Matrix

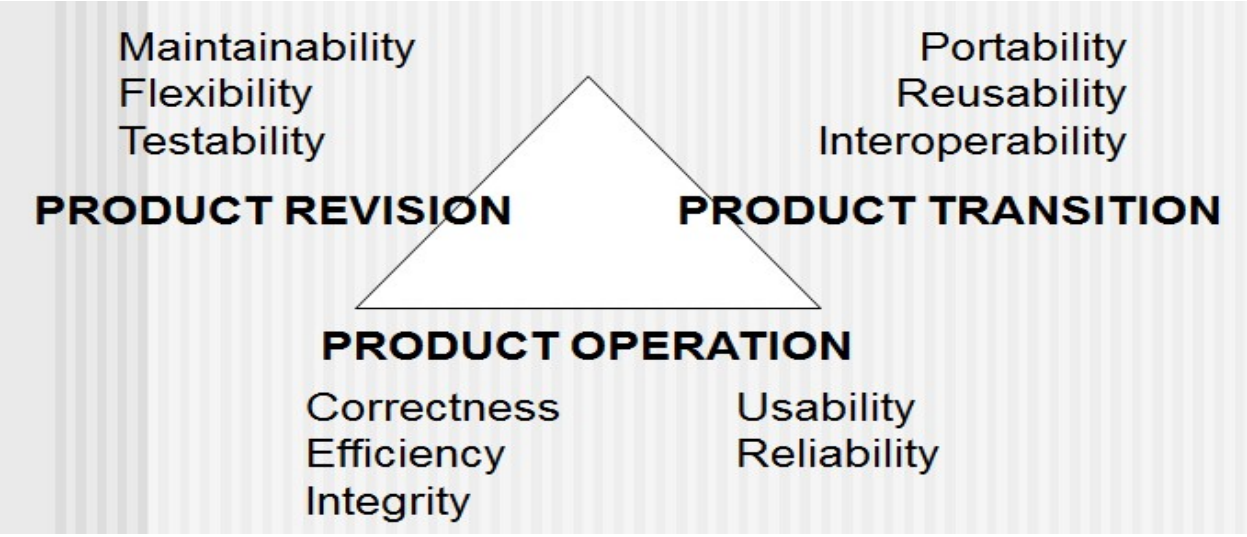
- The **matrix is not symmetrical**:
  - Increasing attribute A may affect B differently than increasing B affects A.
- Example:
  - Boosting **efficiency** may not affect **integrity**,
  - But increasing **integrity** may **hurt efficiency** (e.g., more authentication, encryption).
- Use the matrix to:
  - **Balance quality attributes**
  - **Identify and prioritize** key attributes early
  - **Avoid conflicting goals** (e.g., high usability vs. high portability)
- **Conflicting requirements** can't all be fully satisfied — trade-offs are essential.

Implementing Non-Functional Requirements

- **Quality attributes** (nonfunctional) can result in:
  - **Derived functional requirements**
  - **Design decisions**
  - **Technical specifications**
- Example:
  - A **medical device** needing high availability:
    - **Design** includes backup battery.
    - **Functional requirement**: system must **alert** (visually/audibly) when on battery power.

Quality Attribute Types	Technical Information Category
Integrity, Interoperability, Robustness, Usability, Safety	Functional Requirement
Availability, Efficiency, Flexibility, Performance, Reliability	System Architecture
Interoperability, Usability	Design Constraint
Flexibility, Maintainability, Portability, Reliability, Reusability, Testability, Usability	Design Guideline
Portability	Implementation Constraint

McCall's Triangle of Quality



McCall's Triangle of Quality is a classic software quality model developed by Jim McCall. It categorizes software quality attributes into three key perspectives:

1. Product Operation (How well it works)

Focuses on how the software behaves during use.

- **Correctness** – Extent to which software meets specifications and user needs
- **Reliability** – Ability to perform intended functions under defined conditions
- **Efficiency** – Use of system resources like CPU, memory, bandwidth
- **Integrity** – Protection from unauthorized access and data corruption
- **Usability** – Ease of learning and using the software

2. Product Revision (Ease of change)

Focuses on maintainability and adaptability.

- **Maintainability** – Ease of identifying and fixing defects
- **Flexibility** – Ease of making modifications for changing requirements
- **Testability** – Ease of verifying the software through testing

3. Product Transition (Ease of movement to other environments)

Focuses on portability and adaptability to new settings.

- **Portability** – Ability to operate in different environments
- **Reusability** – Use of software components in other applications
- **Interoperability** – Ability to interact with other systems

This triangle helps identify and balance **quality attributes** during software development. Each vertex represents a different aspect of software quality.

MCQ Practice

1. **How does understanding software quality attributes early in the development process influence architectural decisions?**

a) It delays architectural decisions until after functional requirements are fully implemented.

b) It allows architects to design the system to meet essential quality goals from the start, avoiding costly re-architecting later.

c) It encourages focusing only on functionality, ignoring quality attributes until testing.

d) It primarily helps in selecting programming languages but has little effect on architecture.

2. **Which of the following best describes the concept of software availability?**

a) The speed at which the system processes transactions under load.

b) The degree to which the software recovers from faults without failure.

c) The ease with which new features can be added to the software.

d) The proportion of planned uptime during which the system is fully operational and accessible to users.

3. **In a scenario where a 911 emergency telephone system is overwhelmed with calls, which quality attribute is most relevant to specify how the system should behave?**

a) Performance, particularly how the system's performance degrades under overload conditions.

b) Integrity, to prevent unauthorized access during emergencies.

c) Reusability, ensuring components can be reused in other emergency systems.

d) Usability, focusing on how easy it is for operators to manage calls.

4. **Why is efficiency considered a critical quality attribute in real-time process control systems?**

a) Because efficiency directly measures user satisfaction with the interface design.

b) Because efficiency defines how easy it is to add new capabilities to the system.

c) Because efficiency ensures the software is reusable across different platforms.

d) Because inefficient resource use can cause performance degradation that risks safety in time-critical operations.

5. **How does flexibility influence software design in systems developed incrementally or through evolutionary prototyping?**

a) It allows easy addition of new capabilities and facilitates successive releases with minimal effort.

b) It ensures the software runs efficiently on limited hardware resources.

c) It focuses on maximizing throughput and response times.

d) It guarantees the software will never fail under unexpected conditions.

6. Which statement best captures the essence of software integrity as a quality attribute?

a) It describes how easily the software can be modified and maintained.

b) It measures how quickly the system processes user requests.

c) It defines how well the system interoperates with other software components.

d) It involves protecting the system from unauthorized access, data loss, and ensuring privacy and safety of data.

7. When assessing interoperability requirements, what is the key consideration?

a) Measuring how fast the system responds to user inputs.

b) Identifying which other systems the software must exchange data or services with and understanding expected data formats.

c) Ensuring the software can recover from hardware failures gracefully.

d) Determining how easy it is to modify the software's source code.

8. What is the primary way to measure software reliability?

a) By the speed at which the system processes transactions.

b) By the amount of processor capacity the software uses during peak load.

c) By the probability of executing without failure over a period and the percentage of correctly completed operations.

d) By how easily new features can be added to the software.

9. How does robustness contribute to software quality?

a) By ensuring the software uses minimal memory and processor resources.

b) By enabling the system to continue functioning properly despite invalid inputs, defects, or unexpected conditions.

c) By allowing the software to be reused in multiple applications.

d) By making the software easy to learn and operate for new users.

10. Why is usability a critical quality attribute for desktop systems?

a) Because it determines how many transactions per second the system can handle.

b) Because it specifies how easily the software can be modified by developers.

c) Because it measures the effort required to operate the system and how user-friendly it is, impacting user satisfaction.

d) Because it defines how secure the system is against unauthorized access.

11. In what way is maintainability related to flexibility and testability?

a) Maintainability only concerns the speed of the software, unrelated to flexibility or testability.

b) Maintainability is about how well software exchanges data with other systems, unrelated to flexibility or testability.

c) Maintainability depends on how easily software can be understood, changed, and tested, linking it closely to flexibility and testability.

d) Maintainability focuses exclusively on user interface design, not on flexibility or testability.

12. What characteristics must reusable software components have?

a) They must be modular, well documented, independent of specific applications, and somewhat generic in capability.

b) They should be tightly coupled with other components to maximize efficiency.

c) They must have minimal documentation to reduce development time.

d) They must be optimized for performance in a single application only.

13. Why is testability especially important for software products that undergo frequent modifications?

a) Because testability guarantees the software will never fail in production.

b) Because testability ensures the software runs efficiently under heavy load.

c) Because testability focuses on how easily users can learn the software.

d) Because frequent changes require regression testing to ensure existing functionality is not broken.

14. Which quality attributes are typically most important for embedded systems?

a) Flexibility, reusability, and testability.

b) Efficiency, reliability, safety, installability, and serviceability.

c) Availability, integrity, maintainability, and scalability.

d) Interoperability and usability.

15. What is a key challenge when defining quality attribute requirements for software?

a) Quality attributes are irrelevant compared to functional requirements.

b) Users often do not explicitly state their quality expectations, requiring analysts to interpret terms like 'user-friendly' or 'robust'.

c) Quality attributes never influence architectural decisions.

d) Users always provide precise numeric targets for quality attributes, making analysis straightforward.

16. How can increasing the attribute of integrity affect efficiency in a software system?

a) Increasing integrity reduces the need for testing, thus improving efficiency.

b) Integrity and efficiency are unrelated and do not affect each other.

c) Increasing integrity can decrease efficiency because additional security measures add processing overhead.

d) Increasing integrity always improves efficiency by streamlining processes.

17. Why is it important to identify and prioritize quality attributes during requirements elicitation?

a) Because prioritizing quality attributes delays the development process unnecessarily.

b) Because quality attributes are less important than functional requirements.

c) To avoid conflicting goals that make it impossible to fully satisfy all requirements.

d) Because quality attributes only affect testing and not design.

18. How can non-functional quality attributes lead to derived functional requirements?

a) By only influencing the user interface design without affecting functionality.

b) By focusing exclusively on code style and documentation standards.

c) By replacing all functional requirements with quality attributes.

d) By specifying quality goals that require certain system behaviors or features to achieve them, such as backup power for availability.

19. Which of the following best illustrates a measurable usability requirement?

a) The system shall be available 99.5% of the time on weekdays.

b) A trained user shall be able to submit a complete request in an average of four and a maximum of six minutes.

c) Only users with Auditor access shall view customer transaction histories.

d) The software shall use no more than 75% of processor capacity at peak load.

20. What is the formula for calculating software availability?

a) Availability = Mean Time to Repair / (Mean Time to Failure + Mean Time to Repair)

b) Availability = Mean Time to Failure / (Mean Time to Failure + Mean Time to Repair)

c) Availability = Mean Time to Failure × Mean Time to Repair

d) Availability = Mean Time to Failure - Mean Time to Repair

True/False Practice

1. Availability is calculated as the ratio of mean time to failure (MTTF) to the sum of MTTF and mean time to repair (MTTR).

2. Efficiency and performance are unrelated software quality attributes.

3. Robustness refers to the system's ability to handle invalid inputs and recover gracefully from errors.

4. Usability only concerns how easy it is for expert users to operate the software.

5. Maintainability is closely related to flexibility and testability.

6. Increasing software integrity always improves efficiency.

7. Interoperability measures how easily a system can exchange data or services with other systems.

8. Reusability requires software components to be modular, well documented, and independent of specific applications.

9. Testability refers to the ease with which software can be tested for defects and is also known as verifiability.

10. Quality attributes are only important to software developers, not to users.

PRODUCT METRICS

Ch: 12

Function-Based Metrics

- **Function Points (FP)** measure software **functionality size**.
- Proposed by **Albrecht**.
- Calculated from:
  - Countable **information domain elements**.
  - Assessments of software **complexity**.

Information Domain Elements:

Element	Description
External Inputs (EIs)	Input transactions that update internal files
External Outputs (EOs)	Outputs to users (e.g., reports)
Internal Logical Files (ILFs)	Groups of related data accessed together (e.g., purchase orders)
External Interface Files (EIFs)	Files shared among different applications
External Inquiries (EQs)	Transactions that provide info without updating files

Function Points Metrics

Information Domain Value	Count (X)	Simple FP Value	Average FP Value	Complex FP Value	Total FP Calculation (=)
Number of External Inputs (EIs)	X	3	4	6	Count × corresponding FP value
Number of External Outputs (EOs)	X	4	5	7	Count × corresponding FP value
Number of External Inquiries (EQs)	X	3	4	6	Count × corresponding FP value
Number of Internal Logical Files (ILFs)	X	7	10	15	Count × corresponding FP value
Number of External Interface Files (EIFs)	X	5	7	10	Count × corresponding FP value
Count Total:					**Sum all totals across categories.**

Metrics for OO Design

➔Whitmire's nine measurable characteristics of Object-Oriented (OO) design:

1. Size: Measured by volume, length, and functionality of the design.

2. Complexity: Reflects how interrelated the classes are within the system.

3. Coupling: Describes the degree of interdependence between classes/modules.

4. Cohesion: Measures how well the methods in a class work together toward a single purpose.

5. Sufficiency: Measures if an abstraction/design has all required features; focuses on interface and hiding internals.

6. Completeness: Indicates potential for reuse of an abstraction/design.

7. Primitiveness: Degree to which an operation/class is atomic or basic.

8. Similarity: How alike classes are in structure, function, behavior, or purpose.

9. Volatility: Likelihood of change occurring in a component.

Class Oriented Metrics

Chidamber and Kemerer Metrics:

- **Weighted methods per class (WMC)**: Count of methods/functions in a class.
- **Depth of the inheritance tree (DIT)**: Inheritance tree depth of a class.
- **Number of children (NOC)**: Number of immediate subclasses.
- **Coupling between object classes (CBO /CBC)**: Degree of inter-class coupling.
- **Lack of cohesion in methods (LCOM)**: Measures method cohesion within a class.

Lorenz and Kidd Metrics:

- **Class Size**: Total size of the class (lines, members).
- **Overridden Operations**: Number of methods overridden in a subclass.
- **Added Operations**: Number of new methods introduced in a subclass.

Code Metrics

Halstead's Software Science: A set of metrics based on counting **operators** and **operands** in code to measure software size, complexity, and effort.

Metrics for Testing

- **Testing effort** can be estimated using design metrics.
- **Binder's metrics for OO testability** include:
  - **Lack of Cohesion in Methods (LCOM)**: Low cohesion makes testing harder.
  - **Percent public and protected (PAP)**: % of public/protected members; more exposure = more to test.
  - **Public access to data members (PAD)**: Public data access increases test complexity.
  - **Number of root classes (NOR)**: More root classes = broader testing scope.
  - **NOC & DIT**: More children/deeper trees = more complex inheritance to test.

Maintenance Metrics

- **Purpose**: Indicates **stability** of a software product across releases.
- **Defined Values**:
  - **MT** = Total modules in current release
  - **Fc** = Modules changed in current release
  - **Fa** = Modules added
  - **Fd** = Modules deleted
- **Formula**:

$$\mathbf{SMI} = \frac{\mathbf{MT} - (\mathbf{Fa} + \mathbf{Fc} + \mathbf{Fd})}{\mathbf{MT}}$$

- **Interpretation**:
  - **SMI → 1.0** means **high stability**
  - Lower SMI indicates more changes and less maturity/stability

MCQ Practice

21. Which of the following best describes the purpose of function point metrics in software engineering?

A) Measure amount of code written
B) Assess user satisfaction with interface
C) Estimate functionality delivered based on information domain and complexity
D) Evaluate performance speed of functions

22. In the context of function point metrics, what does 'external inquiries (EQs)' refer to?

A) Transactions updating internal files
B) Files shared between applications
C) Transactions providing information without updating files
D) Data output transactions like reports

23. Which metric is NOT one of the nine characteristics of OO design described by Whitmire?

A) Volatility
B) Cyclomatic complexity
C) Coupling
D) Sufficiency

24. How does 'coupling' affect software quality in OO design?

A) Low coupling means heavy dependency
B) Measures encapsulation
C) High coupling means strong independence
D) High coupling increases complexity and reduces maintainability

25. What does 'lack of cohesion in methods' (LCOM) indicate about a class?

A) Number of child classes
B) Total number of methods
C) Number of unrelated methods indicating poor cohesion
D) Depth of inheritance

26. Which metric helps estimate software stability across releases?

A) Function point count
B) Average operation size
C) Weighted methods per class
D) Software Maturity Index (SMI)

27. Given MT=100, Fc=5, Fa=3, Fd=2, what is SMI?

A) 0.97
B) 0.90
C) 0.92
D) 0.95

28. **Which metric measures the number of functions in a class?**
    A) Weighted methods per class (WMC)
    B) Lack of cohesion in methods (LCOM)
    C) Depth of inheritance tree (DIT)
    D) Coupling between classes (CBC)

29. **How do external interface files (EIFs) differ from internal logical files (ILFs)?**
    A) EIFs are inputs, ILFs are outputs
    B) EIFs updated by inputs, ILFs never accessed
    C) EIFs are queries, ILFs are reports
    D) EIFs shared between apps, ILFs maintained internally

30. **Why is 'sufficiency' important in OO design?**
    A) Counts methods
    B) Counts subclasses
    C) Measures physical connections
    D) Measures if class has required features and hides internals

31. **What does 'volatility' measure in OO design?**
    A) Similarity between classes
    B) Likelihood of future changes
    C) Class size in lines of code
    D) Number of overridden methods

32. **High LCOM value implies what about a class?**
    A) Many subclasses
    B) Few methods
    C) Highly reusable
    D) Performs unrelated tasks, needs refactoring

33. **Which metric estimates testing effort for OO systems?**
    A) Number of external inputs
    B) Average operation size
    C) Lack of cohesion in methods (LCOM)
    D) Software Maturity Index (SMI)

34. **What does depth of inheritance tree (DIT) indicate?**
    A) Number of child classes
    B) Total methods in class
    C) Number of overridden methods
    D) Levels from class to root in inheritance

35. How does 'completeness' relate to software reuse?

A) Counts external inputs

B) Indicates how well a design can be reused

C) Assesses physical connections

D) Measures number of methods

36. Which metric focuses on operation-oriented characteristics?

A) Weighted methods per class

B) Average number of parameters per operation

C) Depth of inheritance tree

D) Lack of cohesion in methods

37. Halstead's metrics are based on counting what?

A) Operators and operands

B) Test cases executed

C) External inputs and outputs

D) Classes and methods

38. Why might function points be preferred over lines of code (LOC)?

A) Function points count lines, more accurate

B) Easier to calculate automatically

C) Measure only code complexity

D) Measure functionality independent of language

39. Why is 'percent public and protected' (PAP) important in testing?

A) Indicates accessible data members affecting testability

B) Measures test cases passed

C) Assesses inheritance complexity

D) Counts public methods

40. Which metric measures the number of children a class has?

A) Number of children (NOC)

B) Depth of inheritance tree (DIT)

C) Weighted methods per class (WMC)

D) Lack of cohesion in methods (LCOM)

SOFTWARE CONFIGURATION MANAGEMENT

Ch: 13

Baseline

- **Baseline** is a configuration management concept for **controlling changes without seriously impeding justifiable change.**
- Defined by IEEE as:

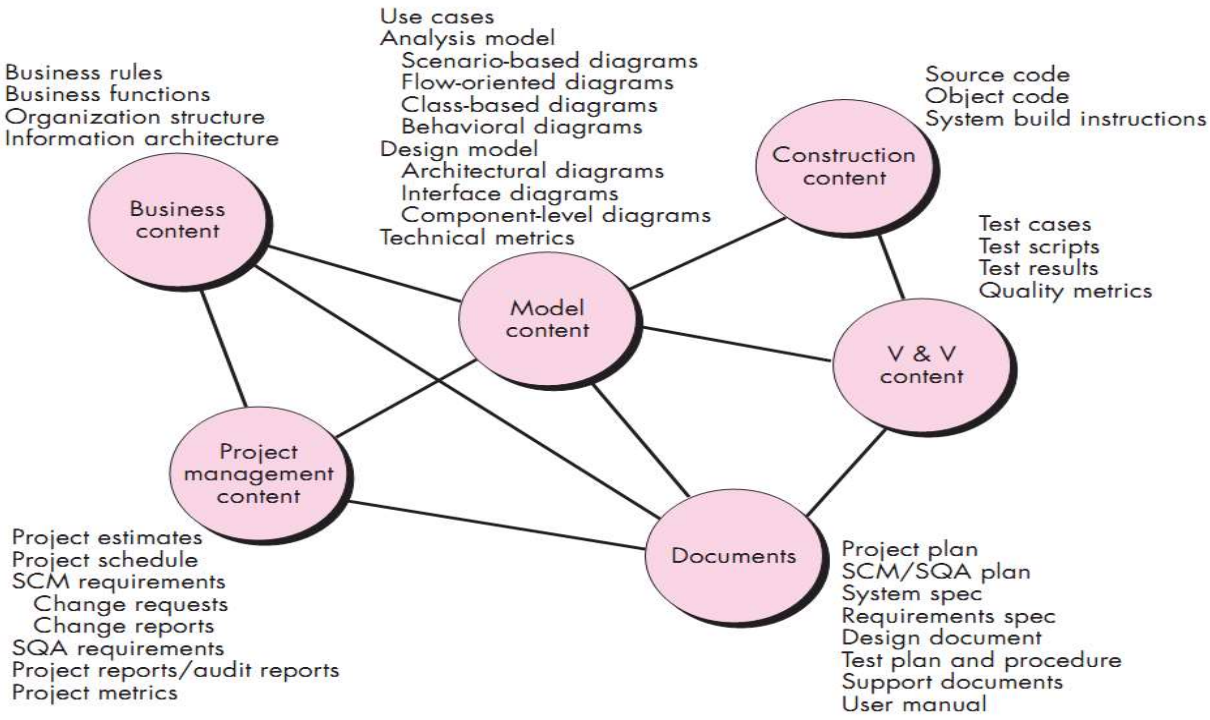
  A formally reviewed and agreed specification/product that serves as a basis for further development.
- Changes to a baseline are allowed **only through formal change control** procedures.
- Example:

  A design model is reviewed, errors are fixed, and once approved, it becomes a **baseline**.

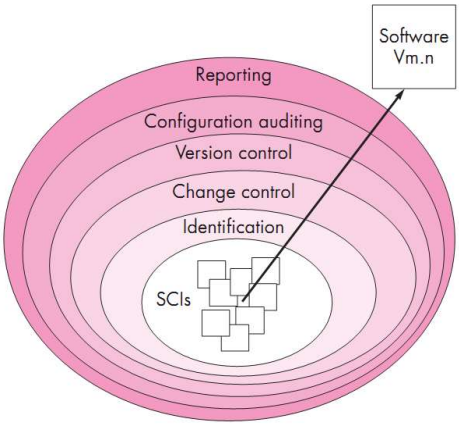
SCM Repository

- **Software configuration**: All items produced during the software development process.
- **SCM repository**: A central database for storing software engineering information.
- **SCM mechanisms**: Tools and structures to manage changes effectively.

Content of SCM Repository



SCM Layers



- **SCIs** flow through layers during their lifetime.
- New SCIs must be **identified**; if unchanged, change control doesn't apply.
- Each SCI is linked to a **specific software version**.
- **Version control** tracks all changes, reasons, authors, and timestamps.
- **Configuration audit** keeps records of SCI details (name, date, version, etc.).

Change Control Process

1. Need for change is recognized
2. Change request from user
3. Developers Evaluates
4. Change report is generated
5. Change control authority decides
6. If (Request is queued for action) …. else (change request is denied)→ user is informed
7. Assign individuals to configuration objects
8. "Check out" configuration objects (items)
9. Make the change (in design)
10. Establish a baseline for testing
11. Perform quality assurance and testing activities (regression testing)
12. Promote changes for inclusion in next release (revision)
13. Rebuild appropriate version of software
14. Review the change to all configuration
15. Include changes in new version
16. Distribute the new version

SOFTWARE PROJECT ESTIMATION

Ch: 14

Software Project Planning

- **Goal**: Create a practical strategy to **control, track, and monitor** the project.
- **Why?**

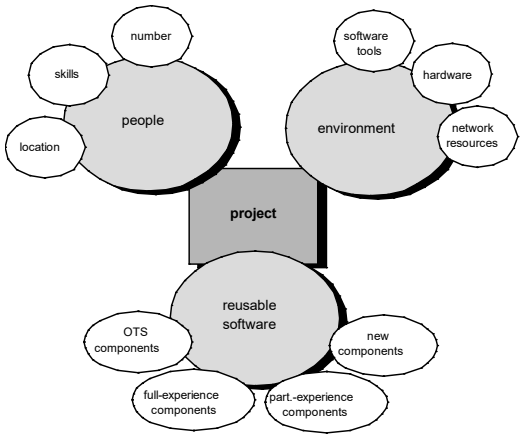
  To ensure the final product is completed **on time**, **within budget**, and with **quality**.

Project Planning Task Set-1

1. **Establish Project Scope** – Define what the project will deliver.
2. **Determine Feasibility** – Check if the project is doable within time, budget, and resources.
3. **Analyze Risks** – Identify potential problems that could affect the project.
4. **Define Required Resources**:
    - **Human Resources** – Determine the number, skills, and location of people.
    - **Reusable Software Resources** – Identify:
        - Off-the-shelf (OTS) components
        - Full-experience and part-experience components
        - New components
    - **Environmental Resources** – Include:
        - Software tools
        - Hardware
        - Network resources

Extra Tip:

Off-the-shelf components are pre-built and ready to reuse as-is.



Project Planning Task Set-2

- **Estimate cost and effort**
  - Decompose the problem
  - Develop two or more estimates using size, function points, process tasks (complexity),
  - Reconcile the estimates

- **Develop a project schedule**
  - Establish a meaningful task set
  - Define a task network
  - Use scheduling tools to develop a timeline chart
  - Define schedule tracking mechanisms

Project Estimation Principles

- Project scope must be understood
- Elaboration (decomposition) is necessary
- Historical metrics (pervious data) are very helpful
- At least two different techniques should be used
- Uncertainty is inherent in the process

Conventional Method

- **Conventional estimation techniques**

- **Task breakdown** and **effort estimation**
- Estimate **size** (e.g., Lines of Code – LOC, Function Points – FP)
- Compute LOC/FP using **information domain metrics**
- Use **historical data** from past projects
- **Automated tools** can support estimation

Effort Estimation

❑ Dynamic Multivariable Effort Estimation Model:

- **Formula:**

$$\mathbf{E} = \left(\frac{\mathbf{LOC} \times \mathbf{B}^{0.333}}{\mathbf{P}}\right)^{3} \times \left(\frac{1}{t^4}\right)$$

Where:

- **E** = Effort (in person-months or person-years)
- **t** = Project duration (in months or years)
- **B** = Special skills factor
- **P** = Productivity parameter
- **LOC** = Estimated lines of code

This model considers **code size**, **skills**, **productivity**, and **time** to estimate total **effort**.

COCOMO (Constructive Cost Model)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

Where:

- **PM**: Effort in person-months
- **SLOC**: Source Lines of Code
- **P**: Complexity factor (1.04 – 1.24)
- **DM**: Project duration (months)
- **T**: Time exponent (0.32 – 0.38)
- **ST**: Average number of people needed

Project Types & Constants:

Type	Coefficient	P	T
Organic	2.4	1.05	0.38
Semi-detached	3.0	1.12	0.35
Embedded	3.6	1.20	0.32

Used to **estimate effort, duration, and staffing** based on **code size and project type**.

COCOMO Project Types

- **Organic**:
  - **Small & simple** projects
  - **Experienced small teams**
  - Example: Showing VUES info on a webpage
- **Semidetached**:
  - **Medium-sized & moderately complex**
  - **Teams with mixed experience**
  - Example: Biometric login data saved in VUES database
- **Embedded**:
  - **Highly complex** projects
  - **Strong hardware-software integration**
  - Example: Biometric devices, elevators

Problem Practice of COCOMO

Problem1: You are tasked with estimating the effort and duration of a hybrid software project using the **Basic COCOMO model**. The system consists of two modules:

- **Module A**: A **library management system** for a small school, handling book listings, issuing, and returns – estimated at **12,000 SLOC**.
- **Module B**: A **barcode scanning and inventory system** for the library, integrated with the book database – estimated at **30,000 SLOC**.

Based on the **COCOMO model parameters**, answer the following questions:

a) Calculate the **effort (PM)** required for each module separately.

b) Compute the **total effort** for the entire project.

c) Estimate the **development time (DM)** based on total effort.

d) Determine the **average staffing (ST)** needed for the project.

➔ **Is given,**

Module A – Organic

- SLOC = 12,000
- Coefficient = 2.4
- P = 1.05
- T = 0.38

Module B – Semi-detached

- SLOC = 30,000
- Coefficient = 3.0
- P = 1.12
- T = 0.35

We Know,

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Module A – Organic

$$\therefore \textbf{Effort (PM)} = 2.4 \times \left(\frac{12{,}000}{1000}\right)^{1.05} = 32.61 \text{ person} - \text{months}$$

For Module B – Semi-detached

$$\therefore \textbf{Effort (PM)} = 3.0 \times \left(\frac{30{,}000}{1000}\right)^{1.12} = 135.36 \text{ person} - \text{months}$$

(b)

⇨ Total Effort:

$$\textbf{Total PM} = 32.61 + 135.36 = 167.97 \text{ person} - \text{months}$$

(c)

⇨ **Development Time (DM):**

➢ Use **T = 0.35** (from semi-detached, as the larger module is semi-detached):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (167.97)^{0.35} = 15.02 \text{ months}$$

(d)

⇨ **Average Staffing (ST):**

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{167.97}{15.02} = 11.18 \approx 12 \text{ persons}$$

Problem2: You are estimating an **online food ordering system** with two components:

- **Component A (Organic)**: Customer-facing web interface – 18,000 SLOC
- **Component B (Semi-detached)**: Admin panel for restaurant owners with order analytics – 27,000 SLOC

Based on the **COCOMO model parameters**, answer the following questions:

a) Calculate the **effort (PM)** required for each module separately.

b) Compute the **total effort** for the entire project.

c) Estimate the **development time (DM)** based on total effort.

d) Determine the **average staffing (ST)** needed for the project.

➔ **Is given,**

Module A – Organic

➢ SLOC = 18,000

➢ Coefficient = 2.4

➢ P = 1.05

- $T = 0.38$

Module B – Semi-detached

- SLOC = 27,000
- Coefficient = 3.0
- P = 1.12
- T = 0.35

We Know,

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{P}$$

For Module A – Organic

$$\therefore \textbf{Effort (PM)} = 2.4 \times \left(\frac{18{,}000}{1000}\right)^{1.05} = 49.91 \text{ person} - \text{months}$$

For Module B – Semi-detached

$$\therefore \textbf{Effort (PM)} = 3.0 \times \left(\frac{27{,}000}{1000}\right)^{1.12} = 120.29 \text{ person} - \text{months}$$

(b)

⇨ Total Effort:

$$\textbf{Total PM} = 49.91 + 120.29 = 170.2 \text{ person} - \text{months}$$

(c)

⇨ **Development Time (DM):**

- Use **T = 0.35** (from semi-detached, as the larger module is semi-detached):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{T}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (170.20)^{0.35} = 15.093 \text{ months}$$

(d)

⇨ **Average Staffing (ST):**

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{170.20}{15.093} = 11.276 \approx 12 \text{ persons}$$

Problem3: You're managing a smart home system with:

- **Component A (Embedded)**: Real-time sensor integration with microcontrollers – 35,000 SLOC
- **Component B (Semi-detached)**: Mobile app to control and monitor devices – 22,000 SLOC

Tasks:

a) Use COCOMO to estimate PM for each component

b) Find total PM, DM, and ST

c) Discuss how embedded systems affect project complexity

➔ **Is given,**

Module A – Embedded

- SLOC = 35,000
- Coefficient = 3.6
- P = 1.20
- T = 0.32

Module B – Semi-detached

- SLOC = 22,000
- Coefficient = 3.0
- P = 1.12
- T = 0.35

We Know,

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Module A – Embedded

$$\therefore \textbf{Effort (PM)} = 3.6 \times \left(\frac{35{,}000}{1000}\right)^{1.20} = 256.55 \text{ person} - \text{months}$$

For Module B – Semi-detached

$$\therefore \textbf{Effort (PM)} = 3.0 \times \left(\frac{22{,}000}{1000}\right)^{1.12} = 95.63 \text{ person} - \text{months}$$

(b)

⇨ Total Effort:

$$\textbf{Total PM} = 256.55 + 95.63 = 352.18 \text{ person} - \text{months}$$

⇨ Development Time (DM):

➢ Use **T = 0.32** (from **Embedded**, as the larger module is **Embedded**):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (352.18)^{0.32} = 16.32 \text{ months}$$

⇨ Average Staffing (ST):

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{352.18}{16.32} = 21.57 \approx 22 \text{ persons}$$

(c)

Embedded systems add complexity due to tight hardware constraints and real-time requirements, increasing development effort.

Problem4: A university project includes:

➔ Module A (Organic): Student registration and grading – 20,000 SLOC

➔Module B (Organic): Faculty management and course scheduling – 15,000 SLOC

Tasks:

a) Compute effort for both modules

b) Total PM and DM

c) Compare this project with a similar semi-detached project in terms of cost and complexity

➔ **Is given,**

Module A – Organic

- SLOC = 20,000
- Coefficient = 2.4
- P = 1.05
- T = 0.38

Module B – Organic

- SLOC = 15,000
- Coefficient = 2.4
- P = 1.05
- T = 0.38

We Know,

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Module A – Embedded

$$\therefore \textbf{Effort (PM)} = 2.4 \times \left(\frac{20{,}000}{1000}\right)^{1.05} = 55.75 \text{ person} - \text{months}$$

For Module B – Semi-detached

$$\therefore \textbf{Effort (PM)} = 2.4 \times \left(\frac{15{,}000}{1000}\right)^{1.05} = 41.21 \text{ person} - \text{months}$$

(b)

⇨ Total Effort:

$$\textbf{Total PM} = 55.75 + 41.21 = 96.96 \text{ person} - \text{months}$$

⇨ Development Time (DM):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (96.96)^{0.38} = 5.68 \text{ months}$$

(c)

Compared to semi-detached, this organic project has lower complexity and cost but may lack scalability.

Problem5: You're developing a health monitoring system:

➔ Single Embedded Module: Includes sensors, real-time alerts, and firmware – 50,000 SLO

Your Tasks is:

a) Calculate PM, DM, and ST using COCOMO

b) Discuss challenges in embedded software development

c) Explain why organic or semi-detached models may not apply

➔ **Is given,**

Module A – Embedded

- SLOC = 50,000
- Coefficient = 3.6
- P = 1.20
- T = 0.32

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Module A – Embedded

$$\therefore \textbf{Effort (PM)} = 3.6 \times \left(\frac{50{,}000}{1000}\right)^{1.20} = 393.61 \text{ person} - \text{months}$$

⇨ Development Time (DM):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (393.61)^{0.32} = 16.91 \text{ months}$$

⇨ Average Staffing (ST):

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{393.61}{16.91} = 23.27 \approx 24 \text{ persons}$$

(b)

Challenges: Hardware constraints, real-time safety, reliability.

(c)

Organic/semi-detached models ignore tight coupling with hardware and regulatory needs.

Problem6: A team is building an e-commerce platform with:

➔Component A (Semi-detached): Seller dashboard and analytics – 28,000 SLOC

➔ Component B (Embedded): Payment system with secure chip-level authentication – 15,000 SLOC

Your Tasks is:

a) Estimate PM for both modules
b) Total effort, duration, and staffing
c) Suggest which module is riskier and why

➔ **Is given,**

Module B – Embedded

- SLOC = 15,000
- Coefficient = 3.6
- P = 1.20
- T = 0.32

Module A– Semi-detached

- SLOC = 28,000
- Coefficient = 3.0
- P = 1.12
- T = 0.35

We Know,

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Module B – Embedded

$$\therefore \textbf{Effort (PM)} = 3.6 \times \left(\frac{15{,}000}{1000}\right)^{1.20} = 92.81 \text{ person} - \text{months}$$

For Module A – Semi-detached

$$\therefore \textbf{Effort (PM)} = 3.0 \times \left(\frac{28{,}000}{1000}\right)^{1.12} = 125.29 \text{ person} - \text{months}$$

(b)

⇨ Total Effort:

$$\textbf{Total PM} = 92.81 + 125.29 = 218.106 \text{ person} - \text{months}$$

⇨ Development Time (DM):

- Use **T = 0.32** (from **Embedded**, as the larger module is **Embedded**):

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (218.106)^{0.32} = 14.005 \text{ months}$$

⇨ Average Staffing (ST):

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{218.106}{14.005} = 15.57 \approx 16 \text{ persons}$$

(c)

Riskier module: Embedded payment system – due to security, hardware, and compliance complexity.

Problem7: BTRC needs to build a new signal processing software for their newest apparatus. The overall size of this software is estimated to be 50,000 lines of code. Using the Basic COCOMO effort estimation model, estimate the following for all three types of projects (Organic, Semi-Detached, and Embedded):

a) **Total effort required (in person-months)**
b) **Approximately how long does the software project take to complete**
c) **How many staff are needed?**

➔ **Is given,**

➢ $\mathbf{SLOC} = \mathbf{50,000}$

We Know,

Type	Coefficient	P	T
Organic	2.4	1.05	0.38
Semi-detached	3.0	1.12	0.35
Embedded	3.6	1.20	0.32

(a)

$$\textbf{Effort (PM)} = \text{Coefficient} \times \left(\frac{\text{SLOC}}{1000}\right)^{\text{P}}$$

For Organic

$$\therefore \textbf{Effort (PM)} = 2.4 \times \left(\frac{50{,}000}{1000}\right)^{1.05} = 145.92 \text{ person} - \text{months}$$

For Semi-detached

$$\therefore \textbf{Effort (PM)} = 3.0 \times \left(\frac{50{,}000}{1000}\right)^{1.12} = 239.86 \text{ person} - \text{months}$$

For Embedded

$$\therefore \textbf{Effort (PM)} = 3.6 \times \left(\frac{50{,}000}{1000}\right)^{1.20} = 393.61 \text{ person} - \text{months}$$

(b)

⇨ Development Time (DM) For Organic:

$$\textbf{Development Time (DM)} = 2.50 \times (\text{PM})^{\text{T}}$$

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (145.92)^{0.38} = 16.60 \text{ months}$$

⇨ Development Time (DM) For Semi-detached:

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (239.86)^{0.35} = 17.01 \text{ months}$$

⇨ Development Time (DM) For **Embedded:**

$$\therefore \textbf{Development Time (DM)} = 2.50 \times (393.61)^{0.32} = 16.91 \text{ months}$$

(c)

⇨ Average Staffing (ST) For Organic:

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{145.92}{16.60} = 8.79 \approx 9 \text{ persons}$$

⇨ Average Staffing (ST) For Semi-detached:

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

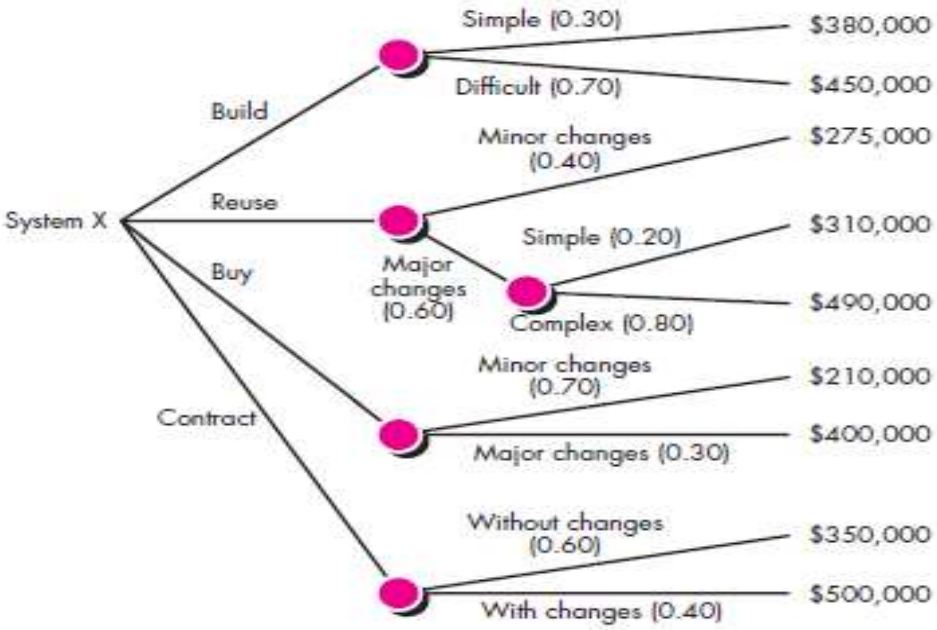
$$\therefore \textbf{Staff Required (ST)} = \frac{239.86}{17.01} = 14.10 \approx 14 \text{ persons}$$

⇨ Average Staffing (ST) For For Embedded:

$$\textbf{Staff Required (ST)} = \frac{\text{PM}}{\text{DM}}$$

$$\therefore \textbf{Staff Required (ST)} = \frac{393.61}{16.91} = 23.27 \approx 23 \text{ persons}$$

Make-Buy Decision



Computing expected cost from decision tree

$$\textbf{expected cost} = (\textbf{path probability})_i \times (\textbf{estimated path cost})_i$$

For example, the expected cost to build is: **expected cost = 0.30 ($380K) + 0.70 ($450K) = $429 K**

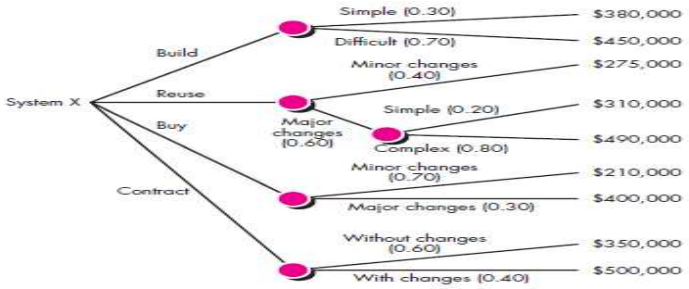
similarly,

expected cost$_{reuse}$ = $382K

expected cost$_{buy}$ = $267K

expected cost$_{contr}$ = $410K

🧠 **Problem1: You're managing a new software project,** System X**, and you face four options:** Build, Reuse, Buy, or Outsource**. Each option has probabilities and estimated costs as follows:**



❓ Your Task is:

a) Calculate the **expected cost** for each option (Build, Reuse, Buy, Outsource).

b) Determine which option offers the **lowest expected cost**.

(a)

We Know,

$$\textbf{expected cost} = (\textbf{path probability})_i \times (\textbf{estimated path cost})_i$$

For Build:

$$\textbf{expected cost} = 0.30 \times 380{,}000 + 0.70 \times 450{,}000 = \$429\text{K}$$

For Reuse:

$$\textbf{expected cost} = 0.40 \times 275\text{K} + 0.60 \times (0.20 \times 310\text{K} + 0.80 \times 490\text{K}) = \$382.4\text{K}$$

For Buy:

$$\textbf{expected cost} = 0.70 \times 210{,}000 + 0.30 \times 400{,}000 = \$267\text{K}$$

For Outsource:

$$\textbf{expected cost} = 0.60 \times 350{,}000 + 0.40 \times 500{,}000 = \$410\text{K}$$

(b)

The **cost** option has the **lowest expected cost** of **$267K**, making it the most cost-effective choice for System X.

PROJECT SCHEDULING

Ch: 15

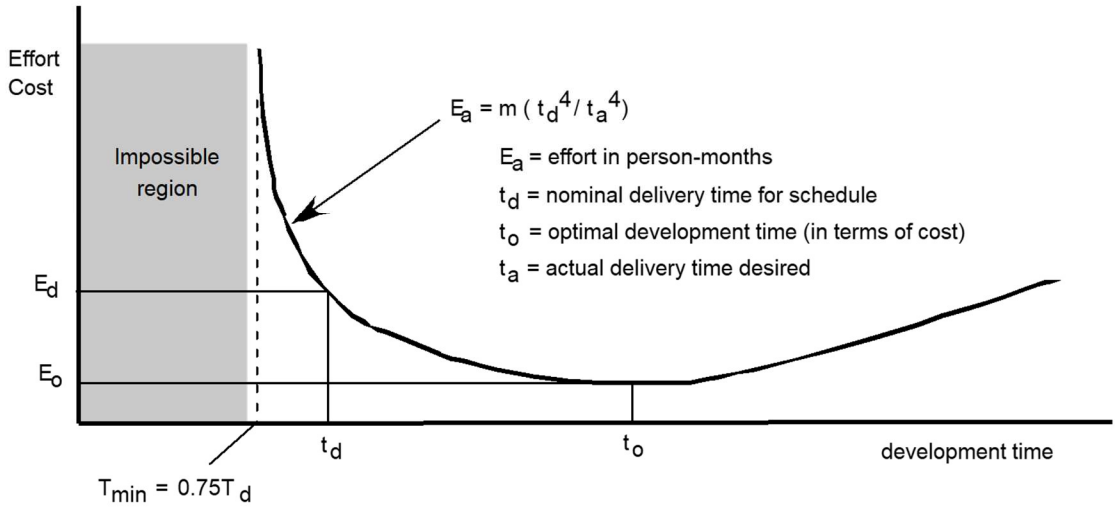
Why are projects late?

- **Unrealistic deadlines** set by outsiders
- **Changing customer requirements** without schedule updates
- **Underestimated effort/resources**
- **Unforeseen risks** (predictable/unpredictable)
- **Unexpected technical issues**
- **Unforeseen human issues**
- **Miscommunication among team members**
- **Poor project tracking and no corrective action**

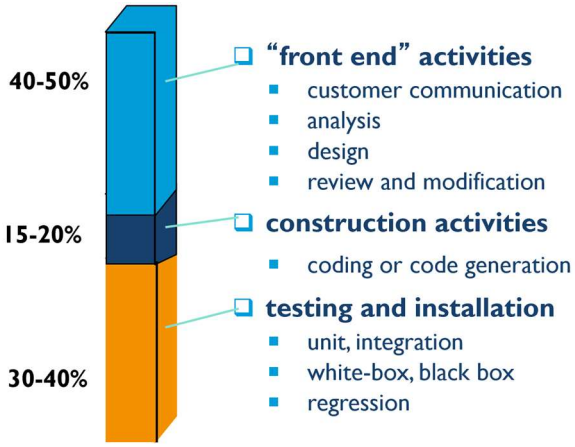
Scheduling principles

- compartmentalization—define distinct tasks
- interdependency—indicate task interrelationship
- effort validation—be sure resources are available
- defined responsibilities—people must be assigned
- defined outcomes—each task must have an output
- defined milestones—review for quality

Effort and delivery time



Effort allocation (40-20-40 rule)



Schedule tracking

- **Hold regular status meetings** for updates and issues
- **Review results** from all engineering process checks
- **Check milestone completion** against schedule
- **Compare actual vs. planned start dates** for tasks
- **Use earned value analysis** for quantitative progress tracking

Earned value analysis (EVA)

- Measures **project progress**
- Assesses **percent complete** using **quantitative data**, not intuition
- Gives **accurate performance insights** as early as **15% into the project**

Computing Earned value

➢ **The budgeted cost of work scheduled (BCWS)** is determined for each work task represented in the schedule.

- **$BCWS_i$** = Planned effort for task *i*
- **BCWS** = ∑ BCWSi for **tasks scheduled Should be completed** by a given time
- **BAC (Budget at Completion)** = Total planned effort for all tasks

→ $\mathbf{BAC} = \sum \mathbf{BCWS_K}$ (for all tasks *k*)

➢ **Budgeted cost of work performed (BCWP):**

- **BCWP** = ∑(BCWS values for **actually completed** tasks)
- Difference from **BCWS**:
  - **BCWS** = Planned work
  - **BCWP** = Completed work

Progress Indicators:

- **Schedule Performance Index (SPI)** $\frac{BCWP}{BCWS}$
  - Measures efficiency of progress vs. plan
- **Schedule Variance (SV)** $= BCWP - BCWS$
  - Shows how ahead or behind schedule we are
- **% Scheduled Complete** $= \frac{BCWS}{BAC}$

  → Work that *should* be done by time *t*
- **% Actual Complete** $= \frac{BCWP}{BAC}$

  → Work that *is actually* done by time *t*

➢ Actual cost for completed tasks (**ACWP):**

Cost Efficiency Indicators:

$$ACWP = \sum(Actual\ cost\ for\ each\ completed\ task)$$

- **Cost Performance Index (CPI)** $= \frac{BCWP}{ACWP}$

  ≫Measures cost efficiency of completed work
- **Cost Variance (CV)** $= BCWP - ACWP$

  ≫Shows if you're over or under budget

🧠 **Problem1:** You are the project manager for a software development project. You've been asked to compute earned value statistics to assess current project performance. The project contains **56 tasks** estimated to require **582 person-days**.

At the point of this analysis:

- **12 tasks** have been completed.
- **15 tasks** were scheduled to be completed by this time.

The planned and actual efforts (in person-days) for the first 15 tasks are given below:

Task	Planned Effort	Actual Effort
1	12.0	12.5
2	15.0	11.0
3	13.0	17.0
4	8.0	9.5
5	9.5	9.0
6	18.0	19.0
7	10.0	10.0
8	4.0	4.5
9	12.0	10.0
10	6.0	6.5
11	5.0	4.0
12	14.0	14.5
13	16.0	—
14	6.0	—
15	8.0	—

a. Calculate the following earned value metrics (show formulas and steps clearly):

i. **BAC** (Budgeted at Completion)

ii. **BCWS** (Budgeted Cost of Work Scheduled)

iii. **BCWP** (Budgeted Cost of Work Performed)

iv. **ACWP** (Actual Cost of Work Performed)

v. **SV** (Schedule Variance)

vi. **CV** (Cost Variance)

vii. **SPI** (Schedule Performance Index)

viii. **CPI** (Cost Performance Index)

b. Based on the values you calculated in part (a), briefly **comment on the project's performance** in terms of cost and schedule.

c. Compute the following project performance indicators:

i. **% Scheduled Complete**

ii. **% Actual Complete**

(a)

(i)

Is given,

Total number of tasks = 56
Total estimated effort = 582 person-days

We Know,

$$\mathbf{BAC} = \sum \mathbf{BCWS_K} \quad \text{(For all tasks } k\text{)}$$

$$\therefore \mathbf{BAC} = \mathbf{582.00}$$

(ii)

We Know,

$$\mathbf{BCWS} = \sum BCWS_i \textbf{ for tasks scheduled Should be completed by a given time}$$

$$\therefore \mathbf{BCWS} = \text{Planned Effort for } \mathbf{all\ 15\ tasks}$$

$$BCWS = 12 + 15 + 13 + 8 + 9.5 + 18 + 10 + 4 + 12 + 6 + 5 + 14 + 16 + 6 + 8$$

$$BCWS = 156.50\ person - days$$

(iii)

We Know,

$$\mathbf{BCWP} = \sum \mathbf{BCWS\ values\ for\ actually\ completed\ tasks}$$

$$\therefore \mathbf{BCWP} = \text{Planned Effort for } \mathbf{completed\ 12\ tasks}$$

$$BCWP = 12 + 15 + 13 + 8 + 9.5 + 18 + 10 + 4 + 12 + 6 + 5 + 14$$

$$BCWP = 126.50\ person - days$$

(iv)

We Know,

$$\mathbf{ACWP} = \sum \textit{Actual cost for each completed task}$$

$$\therefore \mathbf{ACWP} = \text{Actual Effort for the } \mathbf{12 \text{ completed tasks}}$$

$$ACWP = 12.5 + 11 + 17 + 9.5 + 9 + 19 + 10 + 4.5 + 10 + 6.5 + 4 + 14.5$$

$$ACWP = 127.50 \ person - days$$

(v)

We have,

$$\text{BCWP} = 126.50 \ \text{person} - \text{days}$$

$$BCWS = 156.50 \ person - days$$

We Know,

$$\mathbf{SV} = \mathbf{BCWP} - \mathbf{BCWS}$$

$$\therefore \mathbf{SV} = \mathbf{126.5} - \mathbf{156.5}$$

$$SV = -30 \ person - days$$

(vi)

We have,

$$\text{BCWP} = 126.50 \ \text{person} - \text{days}$$

$$ACWP = 127.50 \ person - days$$

We Know,

$$\mathbf{CV} = \mathbf{BCWP} - \mathbf{ACWP}$$

$$\therefore \mathbf{SV} = \mathbf{126.5} - \mathbf{127.5}$$

$$SV = -1 \ person - days$$

(vii)

We have,

$\text{BCWP} = 126.50 \ \text{person} - \text{days}$

$BCWS = 156.50\ person - days$

We Know,

$$\mathbf{SPI} = \frac{\mathbf{BCWP}}{\mathbf{BCWS}}$$

$$\therefore \mathbf{SPI} = \frac{\mathbf{126.50}}{\mathbf{156.50}} = \mathbf{0.808307}$$

$$\therefore SPI = 0.808307$$

(viii)

We have,

$\text{BCWP} = 126.50 \ \text{person} - \text{days}$

$ACWP = 127.50 \ \ person - days$

We Know,

$$\mathbf{CPI} = \frac{\mathbf{BCWP}}{\boldsymbol{ACWP}}$$

$$\therefore \mathbf{CPI} = \frac{\mathbf{126.50}}{\mathbf{127.50}} = \mathbf{0.99}$$

$$\therefore CPI = 0.99$$

(b)

Comment on the Results

- **SV = -30**: Project is **30 person-days behind schedule**.
- **CV = -1**: Project is slightly **over budget**, but the overrun is minimal.
- **SPI = 0.82**: The project is progressing **only 82% as fast** as planned.
- **CPI = 0.99**: For every 1 person-day of work budgeted, **0.99 is delivered**
- **SPI < 1** means work is being completed slower than planned.
- **CPI < 1** means we're spending more than the value we're earning.

(c)

We have,

$\text{BCWS} = 156.50\ person - days$

$\text{BCWP} = 126.50\ \text{person} - \text{days}$

$BAC = 582.00$

We Know,

$\%\ \textbf{Scheduled Complete} = \frac{\textbf{BCWS}}{\textbf{BAC}} \times 100\%$ $\therefore \%\ \textbf{Scheduled Complete} = \frac{156.50}{582.00}$ $= 26.89\%$ $\therefore \%\ \textbf{Scheduled Complete} = 26.89\%$	$\%\ \textbf{Actual Complete} = \frac{\textbf{BCWP}}{\textbf{BAC}} \times 100\%$ $\therefore \%\ \textbf{Actual Complete} = \frac{126.50}{582.00}$ $= 21.73\%$ $\therefore \%\ \textbf{Actual Complete} = 21.73\%$

d. Suggest one corrective action the project manager can take if the project is behind schedule and over budget. (2 Marks)

➔ The project is **behind schedule** and slightly **over budget**. A possible corrective action: Reallocate resources to critical tasks or apply schedule crashing techniques to speed up work without significantly increasing cost. Conduct team performance reviews to identify delays and mitigate risks.

🧠 **Problem2:** You are managing a project which is into **six months of its execution**. You are now reviewing the project status and you have ascertained that project is behind schedule. The **actual cost of Activity A is $2,00,000** and that of **Activity B is $1,00,000**. The **planned value** of these activities is **$1,80,000 and $80,000** respectively. The **Activity A is 100% complete**. However, **Activity B is only 75% complete**. Calculate the schedule performance index and cost performance index of the project on the review date.

Is given,

Tasks	Actual Cost (ACWP)	Planned Value (BCWS)	% Complete
Activity A	**$2,00,000**	**$1,80,000**	**100%**
Activity B	**$1,00,000**	**$80,000**	**75%**
Total:	**$3,00,000**	**$2,60,000**	

We Know,

$$\mathbf{BCWP = BCWS \times \% \ Complete}$$

For Activity A:

$$\therefore \mathbf{BCWP_A = \$1,80,000 \times 100\% = \$1,80,000}$$

For Activity B:

$$\therefore \mathbf{BCWP_B = \$80,000 \times 75\% = \$60,000}$$

$$\therefore \mathbf{Total\ BCWP = \$1,80,000 + \$60,000 = \$2,40,000}$$

$$\therefore \mathbf{SPI} = \frac{\mathbf{BCWP}}{\mathbf{BCWS}} = \frac{\mathbf{\$2,40,000}}{\mathbf{\$2,60,000}} = \mathbf{0.923}$$

$$\therefore \mathbf{CPI} = \frac{\mathbf{BCWP}}{\mathbf{ACWP}} = \frac{\mathbf{\$2,40,000}}{\mathbf{\$3,00,000}} = \mathbf{0.80}$$

Since both SPI and CPI are **less than one**, the project is **behind schedule** and is **experiencing cost overrun.**

🧠 **Problem3:** You are managing a project which is into **five months** of its execution. You are now reviewing the status of the project. The **actual cost of Activity A is $3,00,000** and that of **Activity B is $1,50,000**. The **estimated value of these activities was $2,80,000 and $1,20,000 respectively**. The **Activity A is 90% complete** and **Activity B is only 80% complete**. Calculate the schedule performance index, schedule variance, cost performance index and cost variance of the Project on the review date. And make comments on the project status.

Is given,

Tasks	Actual Cost (ACWP)	Estimated Value (BCWS)	% Complete
Activity A	$3,00,000	$2,80,000	90%
Activity B	$1,50,000	$1,20,000	80%
Total:	$4,50,000	$4,00,000	

We Know,

$$\mathbf{BCWP = BCWS \times \%\ Complete}$$

For Activity A:

$$\therefore \mathbf{BCWP_A} = \$2,80,000 \times 90\% = \$2,52,000$$

For Activity B:

$$\therefore \mathbf{BCWP_B} = \$1,20,000 \times 80\% = \$96,000$$

$$\therefore \mathbf{Total\ BCWP} = \$2,52,000 + \$96,000 = \$3,48,000$$

$\therefore \mathbf{SPI} = \frac{\mathbf{BCWP}}{\mathbf{BCWS}} = \frac{\$3,48,000}{\$4,00,000} = 0.87$	$\therefore \mathbf{SV} = \mathbf{BCWP} - \mathbf{BCWS}$ $SV = \$3,48,000 - \$4,00,000 = -\$52,000$
$\therefore \mathbf{CPI} = \frac{\mathbf{BCWP}}{\mathbf{ACWP}} = \frac{\$3,48,000}{\$4,50,000} = 0.77$	$\therefore \mathbf{CV} = \mathbf{BCWP} - \mathbf{ACWP}$ $CV = \$3,48,000 - \$4,50,000 = -\$1,02,000$
• SPI = 0.87: Behind schedule (87% of planned work done) • CPI = 0.773: Over budget (costs higher than planned)	• SV = -$52,000: Behind schedule by $52,000 worth of work • CV = -$102,000: Cost overrun of $102,000

Since both SPI and CPI are **less than one**, the project is **behind schedule** and is **experiencing cost overrun.** Immediate action is needed to control costs and improve progress to get the project back on track.

RISK MANAGEMENT

Ch: 16

Risk Overview

- **Risk** is the chance of negative consequences from future events.
- It arises when **assumptions** in the project plan are wrong.
- When a risk occurs, it becomes a **problem or issue**.
- Risks are **potential problems** that can affect project success.
- They involve **uncertainty and possible loss**.
- **Risk analysis and management** help the team handle uncertainty.
- Key idea: **Things can go wrong**—plan to **minimize impact**.

Risk Management

❑ Reactive Approach

- Respond to risks **after** they occur.
- **Mitigation**: Plan extra resources to lessen impact.
- **Fix on failure**: Apply resources **when** risk hits.

❑ Proactive Approach

- Perform **formal risk analysis**.
- Address **root causes** of risks.
- Examine **external risk sources**.
- Focus on **change management skills**.

Risk Projection & Building a Risk Table

- **Risk Projection** = Estimating and rating risks.
- Two key factors:
    - **Probability** – Likelihood the risk will occur.
    - **Consequences** – Impact if the risk does occur.

- The **four risk projection activities**,

1. **Probability** – Rate how likely the risk is.
2. **Consequences** – Define what happens if it occurs.
3. **Impact** – Estimate effect on project/product.
4. **Accuracy** – Record how reliable the projection is.

Risk Component & Drivers

- **Performance Risk** – Will the product meet requirements and be usable?
- **Cost Risk** – Will the project stay within budget?
- **Support Risk** – Will the software be easy to maintain and update?
- **Schedule Risk** – Will the project finish on time?

Each risk is rated by impact: **negligible, marginal, critical, or catastrophic**.

Probability-Impact Matrix

		Impact				
		Trivial	Minor	Moderate	Major	Extreme
Probability	Rare	Low	Low	Low	Medium	Medium
	Unlikely	Low	Low	Medium	Medium	Medium
	Moderate	Low	Medium	Medium	Medium	High
	Likely	Medium	Medium	Medium	High	High
	Very likely	Medium	Medium	High	High	High

Risk check list

- **Product Size (PS)** – Risk from the size of software.
- **Business Impact (BU)** – Risk from market or management constraints.
- **Customer Characteristics (CU)** – Risk from customer complexity and communication gaps.
- **Process Definition (PR)** – Risk from poorly defined or followed development processes.
- **Development Environment (DE)** – Risk from tool availability and quality.
- **Technology to be Built (TE)** – Risk from system complexity or new technologies.
- **Staff Size & Experience (ST)** – Risk from team's lack of experience or size.

Building Risk Table – 2

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End-users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	
•				
•				
•				

Impact values:
1—catastrophic
2—critical
3—marginal
4—negligible

The work product is called a Risk Mitigation, Monitoring, and Management Plan (RMMM)

Assessing Risk Impact

- **Risk**: Only 70% of planned reusable components will be usable; rest must be custom-built.
- **Probability**: 80% (likely to happen).
- **Impact**: 18 new components × 100 LOC × \$14/LOC = **\$25,200**.
- **Risk Exposure (RE)**: 0.80 × 25,200 = **\$20,200**.

A framework for dealing with risk - risk management

1. **Risk Identification** – Identify possible risks.
2. **Risk Analysis & Prioritization** – Determine which risks are most serious.
3. **Risk Planning** – Decide how to handle the risks.
4. **Risk Monitoring** – Continuously track and reassess risks.

Risk Identification

- **Checklists** – Use past project experience to identify common (generic) risks.
- **Brainstorming** – Gather stakeholders to share and discuss potential risks.
- **Causal Mapping** – Trace chains of cause and effect (e.g., illness → delay → late delivery).

Boehm's top 10 development risks

1. **Personnel shortfalls**: Hire top talent, training, team building, schedule key staff early.
2. **Unrealistic time/cost estimates**: Use multiple methods, historical data, standardize methods.
3. **Wrong software functions**: Use prototyping, user surveys, formal specs, early manuals.
4. **Wrong user interface**: Prototyping, task analysis, user involvement.
5. **Gold plating**: Requirements scrubbing, prototyping, design to cost.
6. **Late requirement changes**: Change control, incremental development.
7. **Poor external components**: Benchmarking, inspections, contracts, quality checks.
8. **Issues in outsourced tasks**: QA procedures, competitive bidding.
9. **Real-time performance issues**: Simulation, tuning, prototyping.
10. **Technical complexity**: Technical analysis, cost-benefit study, training, prototyping.

Risk planning

- **Risk Prevention/Avoidance**: Change plans to eliminate risk (e.g., extend schedule, reduce scope).
- **Risk Reduction**: Lower risk probability through planning (e.g., use prototyping to reduce late changes).
- **Risk Transfer**: Shift risk impact to others (e.g., outsourcing, insurance).

Risk reduction leverage (RRL)

⇨ **RRL** measures how effective a risk reduction action is.

- **Formula:**

$$RRL = \frac{RE_{before} - RE_{after}}{Cost\ of\ reduction}$$

- Example:
  - Before: 20% chance of $20,000 damage → RE_before = $4,000
  - After: 5% chance → RE_after = $1,000
  - Cost: $1,500
  - RRL = (4000 - 1000) / 1500 = 2
- If **RRL > 1**, the risk reduction is worth doing.

Questions Practice

Ch 09: Software Design

Section A: Multiple Choice Questions

1. Which of the following is not a quality of good software design according to Mitch Kapor?
   a) Firmness
   b) Commodity
   c) Delight
   d) Redundancy

2. Modularity helps in software design because:
   a) It makes code execution faster
   b) It makes the software look better
   c) It breaks the design into manageable parts
   d) It increases the number of bugs

3. A cohesive module should ideally:
   a) Perform multiple tasks simultaneously
   b) Have many interfaces
   c) Do one task with minimal interaction with others
   d) Be tightly coupled with other modules

4. Which is not a typical user interface design error?
   a) Lack of consistency
   b) No guidance or help
   c) Context sensitivity
   d) Too much memorization

5. Which interface design principle states “The time to acquire a target is a function of the distance to and size of the target”?
   a) Readability
   b) Fitt’s Law
   c) Latency Reduction
   d) Consistency

Section B: True/False

6. A monolithic software is easy to understand and maintain.

7. Coupling indicates the interdependence between modules.

8. The interface should communicate the progress of user-initiated activities.

9. The same interaction pattern should never be reused across multiple applications.

10. A well-designed interface should allow users to interrupt or undo actions.

11. Disclosing too much information at once helps in reducing cognitive load.

12. Interface analysis includes studying users, tasks, content, and environment.

13. The user should be required to use OS commands within the application interface.

14. Refactoring changes the external behavior of the system.

15. “Alt-S to save” is an example of consistent interface behavior.

Section C: Matching Questions

A. Modularity **B. Refactoring** **C. Aspect** **D. Cohesion** **E. Coupling** **F. Interface Analysis** **G. Learnability** **H. Progress Bar**

1. Crosscutting concern in software decomposition
2. Measures interdependency among modules
3. Understanding user, tasks, and environment
4. Allows breaking design into distinct logical parts
5. Design that minimizes the learning time
6. Example of communicating status of ongoing activity
7. Redesign without changing external behavior
8. Interface mimics physical world (e.g., print symbol)
9. Enables movement while enforcing navigation conventions
10. Module doing a single task with minimal interaction

Section D: Broad Questions

1. Discuss the key principles of user interface design. Explain how these principles enhance user experience with relevant examples.
2. What is modularity in software design? Describe its importance, benefits, and trade-offs with suitable explanations.
3. Explain the user interface design process. Discuss the different types of analysis involved in designing an effective user interface.

Answer of Section A:

#	Correct Answer	Marks
1	✅ d	2
2	✅ c	2
3	✅ c	2
4	✅ c	2
5	✅ b	2

Answer of Section B:

#	Correct Answer	Marks
6	❌ False	1
7	✅ True	1
8	✅ True	1
9	❌ False	1
10	✅ True	1
11	❌ False	1
12	✅ True	1
13	❌ False	1
14	✅ True	1
15	✅ True	1

Answer of Section C:

A. Modularity	**4**
B. Refactoring	**7**
C. Aspect	**1**
D. Cohesion	**10**
E. Coupling	**2**
F. Interface Analysis	**3**
G. Learnability	**5**
H. Progress Bar	**6**
I. Real World Metaphor	**8**
J. Controlled Autonomy	**9**

Answer of Section D:

Answer 1: Key Principles of User Interface Design

User interface design aims to make interaction easy, efficient, and enjoyable. The key principles include:

- **Place the user in control**: Interfaces should allow users to initiate actions easily, undo mistakes, and customize their experience (e.g., flexible commands, undo buttons).
- **Reduce the user's memory load**: Use visual cues, intuitive shortcuts (e.g., Alt+P to print), meaningful defaults, and progressive disclosure of information to avoid overwhelming the user.
- **Make the interface consistent**: Maintain consistency in layout, controls, and behaviors across screens and applications (e.g., MS Office using similar icons across apps).

- **Use real-world metaphors**: Interfaces should resemble real-life objects (e.g., a printer icon for printing), helping users understand functionality quickly.
- **Ensure readability and feedback**: The system should clearly display system status (e.g., progress bars), readable content, and navigation cues.

These principles improve usability, reduce errors, and increase user satisfaction.

Answer 2: Modularity in Software Design

Modularity refers to dividing a software system into smaller, manageable, and logically distinct parts called modules.

- **Importance**: It simplifies development, testing, and maintenance by focusing on one module at a time.
- **Benefits**: Enhances readability, reusability, and team collaboration. It also supports parallel development and reduces overall complexity.
- **Functional Independence**: Achieved through:
  - **Cohesion** – Each module should perform a single task (high cohesion).
  - **Coupling** – Modules should interact with minimal dependency (low coupling).
- **Trade-offs**: Too many modules increase integration cost, while too few reduce manageability.
- **Cost Impact**: Finding the optimal number of modules helps balance development and integration costs, leading to better software quality.

Modularity is essential for scalable and maintainable software systems.

Answer 3: User Interface Design Process

The user interface design process involves analyzing users, tasks, and display content to build intuitive systems.

- **Interface Analysis**:
  - **Users**: Understand their skills, education, preferences, and work conditions.
  - **Tasks**: Identify main tasks, subtasks, and workflows (e.g., use cases, task models).
  - **Content**: Determine what data is shown and how (text, graphics, summaries).
  - **Environment**: Consider device type, usage context (e.g., mobile vs. desktop).
- **Display Content Analysis**: Decide data placement, color use, and error presentation (e.g., errors in red, dialog boxes).
- **Design Principles Applied**:
  - **Anticipation**: Design predicts the user's next action.
  - **Efficiency**: Optimize work completion with minimal effort.
  - **Learnability**: Ensure quick learning and minimal relearning.

A structured UI design process ensures user satisfaction, error reduction, and improved productivity.

Ch 10: Software Testing

Part A: Multiple Choice Questions

1. What is the main purpose of software testing?
   A. Speed up development
   B. Find errors before delivery
   C. Improve documentation
   D. Evaluate hardware

2. Who is likely to test the system with the aim of finding errors?
   A. Developer
   B. Client
   C. Independent Tester
   D. Analyst

3. What is the focus of validation in V&V?
   A. Building the system correctly
   B. Evaluating product performance
   C. Building the right product
   D. Debugging

4. In white-box testing, which of the following is tested?
   A. Only output behavior
   B. Interface layout
   C. Internal logic and structure
   D. Installation time

5. What is a stub in testing?
   A. A complete application
   B. A lower-level dummy module
   C. A test plan
   D. A requirement

6. What does regression testing ensure?
   A. Faster loading
   B. Compatibility with OS
   C. No new bugs after changes
   D. GUI stability

7. Which integration approach combines all units at once?
   A. Top-down

B. Bottom-up
C. Big Bang
D. Cluster

8. Which testing ensures the system recovers from crashes?
A. Alpha testing
B. Recovery testing
C. Stress testing
D. Smoke testing

9. Which testing focuses on runtime behavior under heavy load?
A. Performance Testing
B. Usability Testing
C. Beta Testing
D. System Testing

10. Cyclomatic complexity helps determine:
A. Code duplication
B. Interface behavior
C. Number of test paths
D. Development time

Part B: True or False

1. Testing always proves the absence of defects.

2. Verification answers the question: "Are we building the product right?"

3. Smoke testing is conducted after the final product is released.

4. In top-down testing, drivers are used instead of stubs.

5. Regression testing can be automated or manual.

6. Independent testers have no knowledge of internal software design.

7. White-box testing ensures every decision is evaluated on both outcomes.

8. Unit testing is usually done after system testing.

9. Performance testing measures user satisfaction.

10. Cyclomatic complexity equals the number of independent paths.

Part C: Matching

1. Alpha Testing 2. Stub 3. Cluster Testing 4. Recovery Testing

5. Performance Testing 6. Smoke Testing 7. White box Testing 8. Black-box Testing

9. Regression Testing 10. Cyclomatic Complexity

A. Dummy module for bottom-up testing
B. Test recovery after failure
C. Class collaboration test
D. Early testing by users
E. Measures speed and resource usage
F. Initial error-check build testing
G. Internal code logic
H. User requirement validation
I. Retesting after modifications
J. Logical path measurement

Part D: Broad Questions

1. Suppose a company has just added a new "Wishlist" feature to their e-commerce website. They also fixed a critical bug in the payment gateway.

(a) Which type of testing should be performed to ensure that the new Wishlist feature did not break any existing functionality and why?

(b) Which type of testing should be performed to quickly verify that the most critical flows of the application (such as login, add to cart, payment) still work after a new deployment and why?

2. Discuss Cyclomatic Complexity. How is it calculated? Why is it important in determining test cases using basis path testing? Also, find the cyclomatic complexity value if there are 8 edges and 5 nodes.

3. A social networking application allows users to upload images. The image file size must not exceed 5 MB, and the file format must be either JPG, PNG, or GIF. Users report that some unsupported file types are being accepted and the application crashes when large files are uploaded. Evaluate how black-box testing could help uncover these issues. What test cases would you create to test the image upload feature?

Answer of Section A:

Q#	Correct Answer
1	✅ B
2	✅ C
3	✅ C
4	✅ C
5	✅ B
6	✅ C
7	✅ C
8	✅ B
9	✅ A
10	✅ C

Answer of Section B:

Q#	Correct Answer
1	❌ F
2	✅ T
3	❌ F
4	❌ F
5	✅ T
6	✅ T
7	✅ T
8	❌ F
9	❌ F
10	✅ T

Answer of Section C:

Q#	Correct Answer
1	✅ D
2	✅ A
3	✅ C
4	✅ B
5	✅ E
6	✅ F
7	✅ G
8	✅ H
9	✅ I
10	✅ J

Answer of Section D:

1. E-commerce Website Scenario

(a)

Type of testing: Regression Testing

Why?

≫ When a new feature (like Wishlist) is added, it might affect existing functionalities.

≫ Regression testing ensures that previously working parts of the application still work after new changes.

≫ It aims to catch unintended side effects introduced by new code.

(b)

Type of testing: Smoke Testing

Why?

≫ Smoke testing checks basic and critical functionalities (e.g., login, add to cart, payment) to ensure that the application is stable enough for further testing.

≫ It is performed after each build or deployment.

≫ Also known as "build verification testing," it provides quick feedback on whether the major flows work.

2. Cyclomatic Complexity

Definition:

≫ Cyclomatic complexity is a software metric used to measure the complexity of a program's control flow.

≫ It helps determine the number of independent paths through the code, which is important for designing test cases.

How is it calculated?

Formula:

$V(G) = E - N + 2P$

Where:

- E = number of edges
- N = number of nodes
- P = number of connected components (usually 1)

Importance in basis path testing:

≫ In basis path testing, test cases are designed to execute each independent path at least once.

≫ Cyclomatic complexity provides the minimum number of test cases needed to achieve complete branch coverage.

≫Helps ensure all possible logical paths are tested.

Example calculation:

Given: E = 8, N = 5, P = 1

V(G) = 8 - 5 + 2*1 = 5

So, the cyclomatic complexity is 5.

3. Image Upload Feature — Black-box Testing

How black-box testing helps:

≫ Black-box testing focuses on functionality without knowing internal code.

≫ Helps find issues in user inputs, file validations, and error handling.

Test cases to uncover the reported issues:

Test Case ID	Description	Expected Result
TC1	Upload a JPG file < 5 MB	Upload successful
TC2	Upload a PNG file < 5 MB	Upload successful
TC3	Upload a GIF file < 5 MB	Upload successful
TC4	Upload a BMP file < 5 MB	Upload rejected with proper error
TC5	Upload a PDF file < 5 MB	Upload rejected with proper error
TC6	Upload a JPG file > 5 MB	Upload rejected with "File too large" error
TC7	Upload a PNG file exactly 5 MB	Upload successful
TC8	Upload a JPG file with corrupted content	Upload rejected with error
TC9	Upload no file and click upload	Show "No file selected" error
TC10	Upload a JPG file with very high resolution but < 5 MB	Upload successful

Ch 11: Software Quality Attributes

Section A: Multiple Choice Questions

1. What do software quality attributes mostly define?

   A. Functional requirements
   B. Design tools
   C. Nonfunctional requirements
   D. Coding syntax

2. What does MTTF stand for?

   A. Mean Test Time Failures
   B. Maximum Time to Fix
   C. Mean Time to Failure
   D. Minimum Time to Failure

3. Which quality attribute includes attributes like response time and throughput?

   A. Usability
   B. Flexibility
   C. Performance
   D. Maintainability

4. Efficiency is closely related to:

   A. Performance
   B. Flexibility
   C. Testability
   D. Usability

5. Which of the following best defines integrity?

   A. User interface consistency
   B. Resistance to hacking and data loss
   C. Availability of help menu
   D. Reduction of software cost

6. Which attribute measures the ease of exchanging data between systems?

   A. Maintainability
   B. Interoperability
   C. Flexibility
   D. Usability

7. Which of the following is an example of a performance requirement?

   A. The software must be easy to use
   B. It should download in 15 seconds over 50 KBps
   C. Only admin can access data
   D. None of these

8. Reusability depends mostly on:

   A. RAM speed
   B. Operating system
   C. Modularity and documentation
   D. Licensing

9. What does the attribute ‘Robustness’ deal with?

   A. Cost estimation
   B. UI design
   C. Handling unexpected conditions
   D. Code duplication

10. Flexibility is also referred to as:

    A. Usability
    B. Scalability
    C. Extensibility
    D. Durability

11. The example "only auditors can see transaction history" relates to:

    A. Availability
    B. Integrity
    C. Usability
    D. Flexibility

12. What metric is used to define testability?

    A. Cyclomatic complexity
    B. Availability rate
    C. Efficiency ratio
    D. None

13. Which of the following affects maintainability the most?

    A. Ease of use
    B. Ease of change and testing

C. Execution speed
D. All of the above

14. Which quality attribute is MOST important for embedded systems?

A. Usability
B. Efficiency
C. Availability
D. Interoperability

15. "Download every web page in 15 seconds" is related to:

A. Reliability
B. Robustness
C. Performance
D. Flexibility

16. Reusable software must be:

A. Memory intensive
B. OS-dependent
C. Modular and generic
D. None

17. If a system consumes too many resources, it lacks:

A. Testability
B. Performance
C. Integrity
D. Flexibility

18. The matrix shown in the slide highlights:

A. System modules
B. Conflicting attributes
C. User personas
D. Process timeline

19. What is NOT a term used synonymously with flexibility?

A. Extendability
B. Expandability
C. Rigidity
D. Augmentability

20. What's the challenge with reusability?

A. Less user-friendly
B. Lower accuracy
C. High development cost
D. Smaller codebase

Section B: True/False

1. Usability refers only to the user interface.
2. Flexibility allows easy addition of new features.
3. Cyclomatic complexity measures lines of code.
4. Testability is less important for frequently modified systems.
5. Integrity ensures only authorized access.
6. Performance requirements include speed and throughput.
7. Reusable code must be modular.
8. Maintainability is unrelated to testability.
9. Robustness is the system's ability to recover from failure.
10. Efficiency means minimal use of system resources.
11. All quality attributes impact each other equally.
12. Interoperability supports system integration.
13. Usability is the same as learnability.
14. Reusability is easy to quantify.
15. High availability means minimum downtime.
16. Flexibility is crucial for iterative development.
17. Every system must be 100% robust.
18. Maintainability also depends on how understandable the code is.
19. A system can be efficient but not usable.
20. Software quality attributes are all functional requirements.

Section C: Matching

1. Availability **2. Efficiency** **3. Integrity** **4. Performance** **5. Usability**

6. Testability **7. Maintainability** **8. Interoperability** **9. Robustness**

10. Reusability **11. Flexibility**

A. Adds new features easily
B. Uses less CPU or memory
C. Prevents unauthorized access
D. Executes tasks quickly
E. Easy to learn and operate
F. Ease of finding defects
G. Easy to fix or improve
H. Exchanges data with other systems
I. Handles errors gracefully
J. Used in other systems
K. System uptime

Section D: Broad Questions

1. Explain the trade-offs involved in software quality attributes. Use the **Attribute Matrix** concept from the slide to justify how increasing one attribute (like usability or integrity) might reduce another (like performance or efficiency). Provide real-world examples.

2. Explain Reliability, Robustness, and Maintenance with example

3. A government education board launches a web-based system to manage national exam results and student data. The system must handle thousands of student records, ensure secure access for authorized users (schools, students, and officials), and support integration with third-party analytics tools.

   Question:
   As a Software Quality Analyst, evaluate the system using **McCall's Triangle of Quality**:

   (a) The system needs updates to align with new grading policies every year. Which **Product Revision** factor is most relevant here and why?
   (b) The system will be reused in other countries with different educational formats. Which **Product Transition** quality factors must be ensured?

Answer of Section A:

Q	Correct Ans
1	c
2	c
3	c
4	a
5	b
6	b
7	b
8	c
9	c
10	c
11	b
12	a
13	b
14	b
15	c
16	c
17	b
18	b
19	c
20	c

Answer of Section B:

Q	Correct Ans	Result
1	F	✔
2	T	✔
3	F	✔
4	F	✔
5	T	✔
6	T	✔
7	T	✔
8	F	✔
9	T	✔
10	T	✔
11	F	✘
12	T	✔
13	F	✘
14	F	✔
15	T	✔
16	T	✔
17	T	✘
18	T	✘
19	T	✘
20	F	✔

Answer of Section C:

No.	Attribute	Matched With	Description
1	Availability	K	System uptime
2	Efficiency	B	Uses less CPU or memory
3	Integrity	C	Prevents unauthorized access
4	Performance	D	Executes tasks quickly
5	Usability	E	Easy to learn and operate
6	Testability	F	Ease of finding defects
7	Maintainability	G	Easy to fix or improve
8	Interoperability	H	Exchanges data with other systems
9	Robustness	I	Handles errors gracefully
10	Reusability	J	Used in other systems
11	Flexibility	A	Adds new features easily

Answer of Section D

1. Trade-offs involved in software quality attributes

Software quality attributes (like usability, performance, security, maintainability, etc.) often conflict with each other. The Attribute Matrix is a conceptual tool used to understand these trade-offs by mapping how improving one attribute may negatively affect another.

- Example trade-offs:

Usability vs. Performance:
To improve usability, we may add more visual elements (animations, dynamic feedback, guided help). However, these extra features consume more resources, increasing load time and reducing performance.
Example: A rich, interactive dashboard in an e-commerce admin portal might feel more intuitive to users but load slower compared to a simpler text-based interface.

Integrity (Security) vs. Efficiency:
Enforcing strict security measures (like strong encryption, multi-factor authentication) increases integrity but can reduce system efficiency (longer login times, higher resource usage).
Example: Online banking apps require secure login with OTPs or biometric checks, slightly reducing the speed of access.

Flexibility vs. Simplicity:
Making a system highly configurable (flexibility) increases complexity for users, reducing usability and maintainability.
Example: Highly configurable enterprise software (like SAP) offers maximum customization but is harder to learn and maintain.

2. Reliability, Robustness, and Maintenance

Reliability:
Definition: The ability of a system to perform its required functions under stated conditions for a specified period of time without failure.
Example: An online flight booking system should consistently allow users to search and book tickets without crashing.

Robustness:
Definition: The degree to which a system can operate correctly even in the presence of invalid inputs or stressful environmental conditions.
Example: A mobile app that still works gracefully when the network connection is slow or temporarily lost (e.g., caching data locally and retrying later).

Maintainability:
Definition: The ease with which a system can be modified to correct faults, improve performance, or adapt to a changed environment.
Example: A university's student management system can be easily updated to support a new grading system or new reporting requirements without major rewrites.

3. Evaluation using McCall's Triangle of Quality

Background of McCall's model:
McCall's Triangle groups quality into:
≫ Product Operation (correctness, reliability, efficiency, integrity, usability)
≫ Product Revision (maintainability, flexibility, testability)
≫ Product Transition (portability, reusability, interoperability)

(a) Updates for new grading policies

Relevant Product Revision factor: Maintainability

Why?
Maintainability is directly about how easily a system can be changed or updated. In this case, new grading rules every year mean developers must modify the grading logic, data structures, and perhaps report formats — so high maintainability is crucial.

(b) Reuse in other countries

Relevant Product Transition factors:
1. Portability: The system should work across different countries' IT infrastructures (e.g., different servers, operating systems).
2. Reusability: Code and design components should be easily reused with minimal modification in other countries.
3. Interoperability: The system should integrate with other local systems (such as national databases, third-party analytics tools).

Ch 12: Product Metrics

I. Multiple Choice Questions

1. Who first proposed the Function Point metric (FP)?

a) Halstead
b) Pressman
c) Albrecht
d) Whitmire

2. What does ILF stand for in function-based metrics?

a) Internal Logic Format
b) Internal Logical Files
c) Integrated Log Files
d) Internal Local Functions

3. Which one is NOT an element in function point metrics?

a) External Inputs
b) External Outputs
c) Internal Variables
d) External Inquiries

4. OO design metric that measures how likely a change will occur is:

a) Sufficiency
b) Volatility
c) Coupling
d) Cohesion

5. Which metric measures the number of functions in a class?

a) DIT
b) WMC
c) LCOM
d) CBC

6. DIT stands for:

a) Depth in Time
b) Depth of Inheritance Tree
c) Data in Tree
d) Design Integration Technique

7. What is cohesion in OO design?

a) The ability to add subclasses
b) Degree of data hiding
c) Degree to which operations work together
d) Degree of class reusability

8. Which metric relates to the average number of parameters per operation?

a) Average Operation Size
b) Operation Complexity
c) Parameters per Method
d) Operation-Oriented Metric

9. What does LCOM measure?

a) Lack of Coupling in Modules
b) Level of Class Operation Metric
c) Lack of Cohesion in Methods
d) Logical Class Operation Measure

10. Which code metric is based on counting operators and operands?

a) Halstead's Metric
b) Whitmire's OO Metric
c) Chidamber & Kemerer Metric
d) Lorenz & Kidd Metric

II. True / False

11. Function points measure the performance speed of software.

12. EIs are input transactions that update internal computer files.

13. Coupling refers to the level of independence among software modules.

14. Cohesion is unrelated to the purpose of operations in a class.

15. DIT indicates how deep a class is in the inheritance hierarchy.

16. PAP stands for Percent of Abstract Parameters.

17. The Software Maturity Index (SMI) reaches 0.0 as software stabilizes.

18. PAD refers to public access to data members.

19. A higher number of children in a class generally increases reuse potential.

20. Operation complexity refers to how operations work together to form functionality.

21. Function-based metrics apply only to procedural programming.

22. SMI uses module count changes between releases.

23. Similarity measures how alike classes are in structure or behavior.

24. The number of overridden operations by a subclass is a class-oriented metric

25. Primitiveness refers to how atomic an operation is.

26. EQs update internal files.

27. Software metrics can guide testing effort estimations.

28. The term “WMC” stands for Weighted Metrics Class.

29. Completeness indicates how easily the abstraction can be reused.

30. External Interface Files (EIFs) are unrelated to inter-application communication.

III. Matching

31. WMC	32. LCOM	33. DIT	34. SMI	35. EO	36. EQ
37. EIF	38. Volatility	39. Maintenance Metric		40. Testing Metrics	

a. Change-prone measure
b. Testing effort estimation
c. Data sharing among applications
d. Measures instability
e. Weighted Methods Per Class
f. Information without update
g. Output to the user
h. Lack of Cohesion in Methods
i. Software Maturity Index
j. Depth of Inheritance Tree

IV. Broad Questions

41. Suppose a software company has developed a **Library Management System** for a national university based on client requirements. The software is now ready for release. As part of the **quality assurance process**, the **Software Maturity Index (SMI)** must be calculated before delivery.

You are given the following data:

- SMI value = 0.982
- Number of modules in the current release = 850
- Number of modules changed= 5
- Number of modules deleted= 3
- Number of modules added=?

a) Using the SMI formula below, calculate the number of modules **added** in this release.

b) Based on the SMI value, explain the **maturity level** of the software and discuss whether it is stable enough to be released to the client.

Answer of Section A:

Qn	Correct Answer
1	✅ c
2	✅ b
3	✅ c
4	✅ b
5	✅ b
6	✅ b
7	✅ c
8	✅ d
9	✅ c
10	✅ a

Answer of Section B:

Qn	Correct Answer
11	❌ False
12	✅ True
13	✅ True
14	❌ False
15	✅ True
16	❌ False
17	❌ False
18	✅ True
19	✅ True
20	✅ True
21	❌ False
22	✅ True
23	✅ True
24	❌ False
25	✅ True
26	❌ False
27	✅ True
28	✅ True
29	✅ True
30	❌ False

Answer of Section C:

Qn	Correct Answer
31	✅ e
32	✅ h
33	✅ j
34	✅ i
35	✅ g
36	✅ f
37	✅ c
38	✅ a
39	✅ d
40	✅ b

41.

Given Data:

- SMI value = 0.982
- Number of modules in the current release (M_t) = 850
- Number of modules changed (Mc) = 5
- Number of modules deleted (M_D) = 3
- Number of modules added (Ma) = ?

a) Calculation of number of modules added

Formula for Software Maturity Index (SMI):

SMI = $(M_t - (M_c + M_a + M_D)) / M_t$

Substituting the given values:

$0.982 = (850 - (5 + M_a + 3)) / 850$

$0.982 = (850 - 8 - M_a) / 850$

$0.982 \times 850 = 850 - 8 - M_a$

$834.7 = 842 - M_a$

$M_a = 842 - 834.7$

$M_a = 7.3 \approx 7$ (rounded to the nearest whole number)

Number of modules added: 7

b) Maturity level analysis

Interpretation:

An SMI value ranges from 0 (very immature) to 1 (highly mature and stable). A value of 0.982 indicates that very few modules were changed, deleted, or added. This means the system is highly stable and mature.

Conclusion:

- The system is highly stable and ready for release.
- Very few modifications suggest minimal risk of new defects.
- The software is mature enough to be confidently delivered to the client.

Ch 14: Software Project Estimation

Section A: Multiple Choice Questions

1. What is the primary goal of software project planning?
   a) Increase complexity
   b) Delay project delivery
   c) Track and monitor the project
   d) Reduce budget only

2. Which of the following is part of the project planning task set-I?
   a) Establish project budget
   b) Define hardware requirements
   c) Determine feasibility
   d) Design UI

3. Which resource is considered “Off-the-shelf”?
   a) Custom-developed module
   b) Software that needs coding
   c) Software available in stock
   d) Cloud infrastructure

4. In estimation, historical metrics refer to:
   a) Documentation history
   b) Data from past projects
   c) Developer experience only
   d) Code written in the past month

5. What is LOC in software estimation?
   a) Language of Code
   b) Lines of Code
   c) Level of Compilation
   d) Local Object Compiler

6. The “E” in the effort estimation formula refers to:
   a) Energy consumed
   b) Employees involved
   c) Effort in person-months/years
   d) Environment variable

7. COCOMO is used to:
   a) Calculate profit
   b) Estimate software cost

c) Debug code
d) Manage HR

8. In COCOMO, “Organic” type projects are:
a) Large and complex
b) Highly regulated
c) Simple and small
d) Hardware-dependent

9. Which of the following is a formula used in decision tree estimation?
a) Expected Cost = Price × Quantity
b) Expected Cost = Σ(Path Probability × Path Cost)
c) Expected Cost = Income – Expenditure
d) None of the above

10. Which of the following is a task in project estimation?
a) Establish UI model
b) Develop a timeline chart
c) Select a compiler
d) Train all users

Section B: True/False

1. Project scope is not required for software estimation.

2. Historical data improves estimation accuracy.

3. Reusable software resources are identified during planning.

4. Elaboration means decomposition.

5. LOC stands for Logical Output Code.

6. The COCOMO model ignores project size.

7. Estimation always gives accurate results.

8. Project planning reduces chances of deadline failure.

9. COCOMO considers complexity of the software.

10. “Make-buy” decisions are irrelevant in software engineering.

Section C: Match the Following

1. Effort (E)
2. LOC
3. Productivity Parameter (P)
4. Project Scope
5. Organic COCOMO
6. Semidetached COCOMO
7. Embedded COCOMO
8. Expected Cost
9. Reusable Resources
10. Task Decomposition

a. 0.30($380K) + 0.70($450K)
b. Lines of code
c. In effort estimation model
d. Must be understood
e. Small, simple projects
f. Mix of hardware/software, intermediate
g. Strongly hardware-dependent projects
h. Decision Tree Formula
i. Off-the-shelf components
j. Project breakdown for estimation

Section D: Broad Questions

1. A local startup company is developing a mobile health monitoring application. The team estimates the project will require around 8,000 lines of code (LOC). The available team has a special skills factor (B) of 1.2, a productivity parameter (P) of 20, and they aim to complete the project within 6 months. As the project manager, you are asked to estimate the total effort (E) in person-months using the following

2. You are working as a consultant for a mid-sized IT company that has been contracted to develop an inventory management system for a national retail chain. The estimated size of the software is 50,000 Source Lines of Code (SLOC). The project team consists of a mix of experienced and less-experienced developers, and the system has a moderate level of complexity.Based on this scenario and using the Basic COCOMO Model, answer the following:

a) Calculate the total **Effort (in Person-Months)** required.
b) Calculate the **Development Time (in Months)**.
c) Determine the **average staffing level** (i.e., number of personnel needed throughout the project).

3. What are the steps in **Project Planning Task Sets I & II**? Why is each step important in real-world project execution?

Answer of Section A:

Q	Correct Answer
1	✅ c
2	✅ c
3	✅ c
4	✅ b
5	✅ b
6	✅ c
7	✅ b
8	✅ c
9	✅ b
10	✅ b

Answer of Section B:

Q	Correct Answer
1	❌ F
2	✅ T
3	✅ T
4	✅ T
5	❌ F
6	✅ T
7	❌ F
8	✅ T
9	✅ T
10	❌ F

Answer of Section C:

Q	Correct Match
1	✅ e
2	✅ b
3	✅ c
4	✅ d
5	✅ a
6	✅ f
7	✅ g
8	✅ h
9	✅ i
10	✅ j

Answer of Section D:

1. Effort Estimation Using Formula

Given:

≫ LOC = 8,000
≫ B (special skills factor) = 1.2
≫ P (productivity parameter) = 20
≫ t (project duration) = 6 months

Formula:
E = ((LOC × B^0.333) / P)^3 × (1 / t^4)

Step-by-step Calculation:
1. B^0.333 ≈ 1.062
2. LOC × B^0.333 = 8000 × 1.062 ≈ 8496
3. 8496 / 20 = 424.8
4. 424.8^3 ≈ 76,540,000
5. 1 / 6^4 = 1 / 1296 ≈ 0.0007716
6. E = 76,540,000 × 0.0007716 ≈ 59.06 person-months

2. COCOMO Estimation

Given:

≫ SLOC = 50,000
≫ Model type: Semidetached
≫ Team: Mixed experience
≫ Complexity: Moderate

Basic COCOMO Formulas:
≫ Effort (PM) = 3.0 × (SLOC / 1000)^1.12
≫ Development Time (T) = 2.5 × (Effort)^0.35
≫ Staffing = Effort / Time

a) Effort:
PM = 3.0 × (50)^1.12 ≈ 3.0 × 78.37 ≈ 235.11 person-months

b) Development Time:
T = 2.5 × (235.11)^0.35 ≈ 2.5 × 6.75 ≈ 16.88 months

c) Staffing:
Staffing = 235.11 / 16.88 ≈ 14 people

3. Project Planning Task Sets I & II

Task Set I – Early Planning Steps

1. Establish Project Scope – Defines boundaries and objectives.
2. Determine Feasibility – Assesses whether project goals are achievable.
3. Analyze Risks – Identifies risks to mitigate them early.
4. Define Required Resources – Ensures necessary tools and infrastructure.
5. Determine Required Human Resources – Identifies skillsets and roles.
6. Define Reusable Software Resources – Promotes efficiency and reuse.
7. Identify Environmental Resources – Prepares technical and physical environment

Task Set II – Estimation & Scheduling

1. Estimate Cost and Effort – Forecasts time and budget.
2. Decompose the Problem – Breaks project into manageable units.
3. Develop Multiple Estimates – Increases reliability through comparisons.
4. Reconcile Estimates – Finalizes a realistic project plan.
5. Develop Project Schedule – Outlines timelines for completion.
6. Define a Meaningful Task Set – Groups activities logically.
7. Define Task Network – Visualizes dependencies and sequences.
8. Use Scheduling Tools – Automates schedule creation and updates.
9. Define Schedule Tracking Mechanism – Allows project monitoring.

Importance in Real-World Projects

≫ Improves clarity and reduces ambiguity in execution.
≫ Prevents budget overruns and missed deadlines.
≫ Supports better communication with stakeholders.
≫ Facilitates proactive risk management.
≫ Enables efficient use of team and tools.

Ch 15: Project Scheduling

Part A: Multiple Choice Questions

1. Which of the following is not a typical reason why software projects are late?
    a) Unrealistic deadlines
    b) Predictable risks
    c) Sufficient resources
    d) Miscommunication
2. What is the first principle of scheduling?
    a) Effort validation
    b) Compartmentalization
    c) Defined outcomes
    d) Schedule tracking
3. What percentage of effort is allocated to front-end activities in the 40-20-40 rule?
    a) 20%
    b) 30%
    c) 40%
    d) 60%
4. Which of the following is used to track quantitative progress in a project?
    a) Gantt Chart
    b) Status report
    c) Earned Value Analysis
    d) Resource table
5. If BCWP is 100 and ACWP is 80, what is the Cost Performance Index (CPI)?
    a) 1.25
    b) 0.80
    c) 1.00
    d) 0.75
6. What does SPI = BCWP / BCWS represent?
    a) Project cost efficiency
    b) Project schedule efficiency
    c) Resource allocation
    d) Testing completion
7. What is the formula for Schedule Variance?
    a) BCWP – ACWP
    b) ACWP – BCWS
    c) BCWS – BCWP
    d) BCWP – BCWS

8. If BCWP = 150 and BAC = 600, what is the % complete?
    a) 15%
    b) 25%
    c) 30%
    d) 35%
9. Which scheduling element ensures each task has an output?
    a) Milestone
    b) Effort validation
    c) Defined outcomes
    d) Interdependency
10. What is the total planned effort referred to in Earned Value Analysis?
    a) BCWS
    b) BAC
    c) ACWP
    d) SPI

Part B: True/False

1. Earned Value Analysis is based on subjective judgment.

2. BCWP refers to the budgeted cost of work that has actually been completed.

3. SPI less than 1 indicates the project is ahead of schedule.

4. Cost Variance = BCWS - ACWP.

5. A project milestone has no effect on quality review.

6. Actual Cost of Work Performed is also known as ACWP.

7. In 40-20-40 effort rule, 20% is allocated for testing and installation.

8. Miscommunication among team members can lead to project delays.

9. CPI less than 1 indicates cost overrun.

10. Defined responsibilities mean tasks are broken down into smaller components.

Part C: Matching (20 Marks)

A. BAC B. BCWS C. BCWP D. ACWP E. SPI F. CPI

G. SV H. CV I. % Complete J. % Scheduled for Completion

1. Budget at Completion
2. Work scheduled/planned
3. Earned value (completed work's budgeted cost)
4. Total effort actually expended
5. Efficiency of scheduled resource usage
6. BCWP / ACWP
7. BCWP – BCWS
8. BCWP – ACWP
9. BCWP / BAC
10. BCWS / BAC

Part D: Broad Questions

1. Explain the project scheduling principles and why they are critical to successful software project execution. Give real-life scenarios where possible.

2. You're working on a software development project with a Budget at Completion (BAC) of 800 person-days. By the end of day 30, your schedule shows BCWS = 200, BCWP = 160, and ACWP = 170. Based on this data:

- Calculate the SPI, SV, CPI, CV, % Complete, and % Scheduled for Completion.
- Interpret the results and explain what actions you would take based on these indicators to manage the project moving forward.

3. You have been hired by a company that has experienced multiple failed software projects due to poor planning and unclear roles. As the new project manager, outline the essential scheduling principles you would implement and explain how each helps ensure successful project completion. Use real-life or hypothetical situations to justify your approach.

Answer of Section A:

Q	Correct Answer
1	c ✅
2	b ✅
3	c ✅
4	c ✅
5	a ✅
6	b ✅
7	d ✅
8	b ✅
9	c ✅
10	b ✅

Answer of Section B:

No	Correct Answer
1	❌ F
2	✅ T
3	❌ F
4	✅ F
5	❌ F
6	✅ T
7	❌ F
8	✅ T
9	✅ T
10	❌ F

Answer of Section C:

Item	Your Answer
A	1
B	2
C	3
D	4
E	5
F	6
G	7
H	8
I	9
J	10

Answer of Section D:

Q1. Project Scheduling Principles and Their Importance

- **compartmentalization—define distinct tasks**
- **interdependency—indicate task interrelationship**
- **effort validation—be sure resources are available**
- **defined responsibilities—people must be assigned**
- **defined outcomes—each task must have an output**
- **defined milestones—review for quality**

Importance:
These principles ensure timely delivery, cost control, quality outcomes, and effective teamwork.

Example:
A mobile app project sets milestones for prototype, alpha, and final release. Early progress checks help fix delays before they grow.

Q2. Project Metrics Calculation & Interpretation

Given: BAC=800, BCWS=200, BCWP=160, ACWP=170 at day 30

- SPI = 160/200 = 0.8 (Behind schedule)
- SV = 160 - 200 = -40 (Schedule delay)
- CPI = 160/170 ≈ 0.94 (Over budget)
- CV = 160 - 170 = -10 (Cost overrun)
- % Complete = (160/800)*100 = 20%
- % Scheduled = (200/800)*100 = 25%

Interpretation:
Project is behind schedule and slightly over budget.

Actions:
Investigate causes, reallocate resources to critical tasks, control costs, update stakeholders, and monitor more closely.

Q3. Scheduling Principles to Fix Past Failures

- **compartmentalization—define distinct tasks**
- **interdependency—indicate task interrelationship**
- **effort validation—be sure resources are available**
- **defined responsibilities—people must be assigned**
- **defined outcomes—each task must have an output**
- **defined milestones—review for quality**

Example:
Previously, testing delays happened because roles weren't defined. Now, QA is clearly responsible, preventing delays.

Ch 16: Risk Management

Part A: Multiple Choice Questions

1. What is the main goal of risk management in software development?
a. Avoid all possible risks
b. Eliminate coding bugs
c. Understand and manage uncertainty
d. Increase team size

2. Which risk management strategy deals with risks after they occur?
a. Proactive
b. Preventive
c. Reactive
d. Avoidance

3. Which of the following is not a major risk component?
a. Performance Risk
b. Schedule Risk
c. Communication Risk
d. Support Risk

4. What does the term Risk Exposure (RE) represent?
a. Only the cost of mitigation
b. Probability × Consequences
c. Sum of risks
d. The total number of risks identified

5. What is the purpose of the risk checklist?
a. To identify risks based on past experiences
b. To analyze testing strategies
c. To allocate budget
d. To eliminate bugs

6. The process of rating each risk based on probability and consequences is called:
a. Risk identification
b. Risk transfer
c. Risk projection
d. Risk elimination

7. What is the full form of RMMM?
a. Risk Mapping and Mitigation Module
b. Risk Mitigation, Monitoring, and Management
c. Risk Measure and Maintenance Model
d. Risk Mapping & Monitoring Mechanism

8. Which of the following is a technique for risk identification?
a. Unit testing
b. Code refactoring
c. Causal mapping
d. Version control

9. Boehm's Top 10 Risks are related to:
a. Deployment stage
b. Software development process
c. Customer service
d. Financial auditing

10. Risk Reduction Leverage (RRL) helps to:
a. Eliminate low risks
b. Skip risk planning
c. Decide if risk reduction is worth the cost
d. Reduce the number of risks

Part B: True/False

1. ___ Risk is the certainty that an event will occur.

2. ___ Schedule risk deals with uncertainty in project timeline.

3. ___ A critical risk always has catastrophic consequences.

4. ___ Risk management includes both proactive and reactive strategies.

5. ___ Probability-impact matrix helps visualize risk severity.

6. ___ Risk avoidance and risk transfer are both risk planning strategies.

7. ___ The goal of risk reduction is to completely remove the risk.

8. ___ The larger the staff size, the lower the risk always.

9. ___ Product size affects risk levels in software development.

10. ___ Brainstorming is not useful in risk identification.

Part C: Matching

A. Performance Risk B. Schedule Risk C. Cost Risk

D. Support Risk E. Business Impact F. Process Definition

G. Customer Characteristics H. Product Size I. Development Environment

J. Technology to be Built

1. Risk linked to project timeline and delivery
2. Risk due to the ease of fixing, upgrading, or adapting the software
3. Risk of the product not meeting requirements
4. Risk of budget overrun
5. Risk from constraints by management/market
6. Risk from unclear or loosely followed development processes
7. Risk from how easily developers can communicate with clients
8. Risk from software scale (lines of code/modules)
9. Risk from the availability and quality of tools
10. Risk from system complexity and new technologies

Part D: Broad Questions

1. Imagine you are managing a software project that involves building a complex healthcare management system. Describe how you would apply the risk management framework from identification to monitoring. Give real-life examples for each stage (e.g., identification, prioritization, planning, and monitoring).

2. A team is using 60 reusable components in a system. Due to constraints, only 70% will be usable. Given each component is 100 LOC and each LOC costs $14 to build, calculate the risk exposure. Also explain how this scenario demonstrates risk impact and mitigation planning.

3. Discuss and compare Proactive vs. Reactive risk strategies in software engineering. Include their definitions, advantages, drawbacks, and when each is more appropriate. Use practical scenarios to support your explanation.

Answer of Section A:

Q	Correct Ans
1	c
2	c
3	c
4	b
5	a
6	c
7	b
8	c
9	b
10	c

Answer of Section B:

Q	Correct Ans
1	F
2	T
3	F
4	T
5	T
6	T
7	F
8	F
9	T
10	F

Answer of Section C:

Component (A–J)	Correct Match
A	3
B	1
C	4
D	2
E	5
F	6
G	7
H	8
I	9
J	10

Answer of Section D:

Question 1:

1. **Risk Identification**
   - Use **checklists** to identify risks like data breaches, complex integrations, and third-party API failure.
   - Example: Integration with hospital lab APIs may fail due to inconsistent data format.
2. **Risk Analysis & Prioritization**
   - Assess **likelihood** and **impact** (e.g., data breach = high impact & medium likelihood).
   - Use a **probability-impact matrix** to rank risks.
3. **Risk Planning**
   - Prepare mitigation plans:
     - For API failure: create a fallback cache or mock API.
     - For data breach: enforce encryption and access control.
   - Choose strategies like **risk reduction** or **risk transfer** (e.g., cloud security managed by AWS).
4. **Risk Monitoring**
   - Track risks through regular review meetings and **logs**.
   - Use tools to update risk status (e.g., Jira risk board).
   - Example: If risk of delay rises, adjust sprints or add resources.

Question 2:

1. **Risk Description**
   - Only **70% of 60 = 42** components are usable.
   - So, **18 components** must be custom developed.
2. **Risk Impact Calculation**
   - Each component = 100 LOC
   - Cost per LOC = $14
   - Total impact = 18 × 100 × 14 = **$25,200**
3. **Risk Probability**
   - Given = 80%
   - **Risk Exposure (RE)** = 0.80 × $25,200 = **$20,160**
4. **Explanation of Risk Impact**
   - Cost, time, and resource allocation are significantly affected.
   - Unplanned development may delay the schedule.
5. **Mitigation Planning**
   - Start parallel development of critical components.
   - Reduce reliance on failed reusable modules.
   - Allocate buffer budget or developer resources.

Question 3:

Aspect	Proactive Strategy	Reactive Strategy
Definition	Plans risks **before** they happen	Responds **after** risk occurs
Example	Identify possible delays and buffer the schedule	Fix code after sudden API change
Advantages	Reduces surprises; better planning	Saves effort if risk doesn't occur
Drawbacks	Time-consuming; may over-plan	May cause panic, unplanned cost
When to Use	Long-term, high-budget projects	Small projects with flexible deadlines

- **Proactive Example**: Planning backups if cloud service fails.
- **Reactive Example**: Hiring extra developers when deadline is missed.